



Université Cheikh Anta Diop de Dakar
Faculté des Sciences et Techniques
Département Mathématiques et Informatique



Laboratoire d'Algèbre, de Cryptographie,
de Géométrie Algébrique et Applications
LACGAA

Licence 3 — Transmission de Données et Sécurité de l'Information

JOKKO AGRO

UNE PLATEFORME BLOCKCHAIN-IA POUR
LA CERTIFICATION ET LA MONÉTISATION
DES PRODUITS AGRICOLES

Présenté et soutenu par :

Die Mbaye
Mouhamad Al Amine Ba

Sous la direction de :

Dr Aminata Ngome Ba

Jury

Président : Dr Aminata Ngome BA UCAD

Membres : Mme Nafy Dione UCAD

M. Habib Ndiaye UCAD

M. Ndiagua UCAD

Année académique 2024–2025

Dédicaces & Remerciements



Bismillahi Rahmani Rahim, au nom de Dieu le Tout Miséricordieux, le Très Miséricordieux.

Nous dédions ce mémoire :

- ★ À nos très chers **parents**. Puisse Allah nous protéger, vous accorder santé, bonheur et longue vie, et fasse en sorte que nous ne vous décevions jamais.
- ★ À nos chers **frères et sœurs**. Puisse Allah vous garder, éclairer votre route et vous aider à réaliser vos rêves les plus chers.
- ★ À nos **oncles et tantes**. Que le Tout-Puissant vous accorde une longue vie.
- ★ À nos **amis, frères et camarades de promotion**. Qu'Allah SWT nous facilite la suite de notre parcours.

AL HAMDOULILLAH RABBI AL ALAMINE

Nous tenons à exprimer notre profonde gratitude :

- ♡ À nos **parents**, pour votre amour inconditionnel, votre soutien indéfectible et vos prières. Que Dieu vous comble de bénédictions et vous accorde une longue vie.
- ♡ À notre **encadrante Madame Ba Aminata**, pour son accompagnement, sa patience et ses encouragements constants tout au long de ce travail.

merci!

Résumé

Jokko Agro est une plateforme numérique agricole innovante conçue pour répondre aux défis du secteur agricole sénégalais. Elle combine quatre axes technologiques complémentaires : l'Intelligence Artificielle (IA) pour un assistant agricole bilingue français/Wolof avec reconnaissance vocale, la Blockchain Ethereum pour la certification décentralisée et immuable des produits agricoles, les Agricoins (AGC) comme monnaie numérique interne ($1 \text{ AGC} = 100 \text{ FCFA}$) avec un système de paiement hybride et un datawarehouse d'analyse, ainsi qu'une architecture de sécurité multicouche garantissant l'intégrité des données.

La plateforme s'adresse aux agriculteurs et aux acheteurs, leur permettant de certifier les produits via 7 checkpoints blockchain, d'effectuer des transactions sécurisées avec mécanisme d'escrow, et d'accéder à des conseils agronomiques personnalisés. L'architecture technique repose sur Angular 17 pour le frontend, Firebase/Firestore pour le backend, Ethereum Sepolia pour la blockchain, et IPFS pour le stockage décentralisé.

Les résultats attendus incluent une meilleure traçabilité des produits agricoles, une inclusion financière via les AGC, et un accès démocratisé à l'expertise agronomique grâce à l'IA multilingue. Ce mémoire présente l'architecture technique détaillée de la solution et les choix de conception retenus.

Mots-clés : Blockchain, Intelligence Artificielle, Agriculture, Certification, Monnaie numérique, Sénégal, Wolof, Traçabilité, Ethereum, Angular, Firebase.

Abstract

Jokko Agro is an innovative digital agricultural platform designed to address the challenges of the Senegalese agricultural sector. It combines four complementary technological pillars : Artificial Intelligence (AI) for a bilingual French/Wolof agricultural assistant with voice recognition, Ethereum Blockchain for decentralized and immutable certification of agricultural products, Agricoins (AGC) as an internal digital currency (1 AGC = 100 FCFA) with a hybrid payment system and analytical data warehouse, as well as a multi-layer security architecture ensuring data integrity.

The platform targets farmers and buyers, enabling them to certify products through 7 blockchain checkpoints, conduct secure transactions with an escrow mechanism, and access personalized agronomic advice. The technical architecture relies on Angular 17 for the frontend, Firebase/Firestore for the backend, Ethereum Sepolia for the blockchain, and IPFS for decentralized storage.

Expected outcomes include improved traceability of agricultural products, financial inclusion through AGC, and democratized access to agronomic expertise through multilingual AI. This thesis presents the detailed technical architecture of the solution and the design choices made.

Keywords : Blockchain, Artificial Intelligence, Agriculture, Certification, Digital Currency, Senegal, Wolof, Traceability, Ethereum, Angular, Firebase.

Table des matières

Table des Figures	v
Liste des Abréviations	vii
Introduction Générale	1
1 Présentation Générale du Projet	2
1.1 Contexte et état des lieux du secteur agricole sénégalais	2
1.1.1 Caractéristiques structurelles des exploitations	3
1.1.2 Une infrastructure numérique paradoxalement favorable	3
1.2 Problématique et question centrale	3
1.3 Objectifs du projet	4
1.3.1 Objectif général	4
1.3.2 Objectifs spécifiques	5
1.4 Approche globale : une réponse intégrée en quatre axes	5
1.5 Présentation synthétique de la plateforme	6
1.5.1 Acteurs de la plateforme	6
1.5.2 Organisation du mémoire	6
2 État de l'art	7
2.1 Méthodologie d'analyse	7
2.2 Analyse des solutions existantes	7
2.2.1 Analyse détaillée des plateformes	8
2.3 Lacunes des systèmes existants	9
2.4 Positionnement stratégique de Jokko Agro	9
2.5 Valeur ajoutée et innovations	10
2.6 Synthèse et conclusion du chapitre	10
3 Technologies et Outils Utilisés	11

3.1	Architecture technique globale	11
3.2	Frontend : Angular 17	12
3.3	Backend : Firebase / Firestore	12
3.4	Intelligence Artificielle	13
3.4.1	TensorFlow.js — Vision par ordinateur côté client	13
3.4.2	Claude API (Anthropic) — Chatbot agricole multilingue	14
3.4.3	Claude Vision — Analyse d'images de cultures	15
3.4.4	Web Speech API — Reconnaissance vocale pour les langues locales	16
3.4.5	Schéma d'interaction des services IA	17
3.4.6	Synthèse des technologies IA	18
3.5	Blockchain : Ethereum & Smart Contracts	18
3.5.1	Flux de certification blockchain	19
3.6	Services externes	19
3.6.1	Alchemy — Provider RPC	19
3.6.2	Wave et Orange Money — Mobile money	20
3.7	Outils de développement	20
4	Conception et Architecture du Système	21
4.1	Architecture globale du système	21
4.2	Architecture modulaire	21
4.3	Diagramme de classes UML	22
4.3.1	Description des entités principales	23
4.4	Diagrammes de séquence	23
4.4.1	Séquence Blockchain	24
4.4.2	Séquence IA	25
4.4.3	Séquence AGC et Paiement	26
4.5	Flux de données	26
4.5.1	Schéma des flux de données	26
4.6	Base de données Firestore	27
4.6.1	Indexation et requêtes	27
4.7	Scalabilité et déploiement	27

4.8	Sécurité et authentification	28
4.9	Synthèse du chapitre	29
5	Axe Intelligence Artificielle	30
5.1	Présentation de l'axe IA	30
5.2	Chatbot agricole intelligent	30
5.3	Reconnaissance vocale (Wolof)	31
5.4	Analyse d'images	32
5.5	Géolocalisation intelligente	33
5.6	Apports de l'IA dans le système	34
6	Axe Blockchain et Certification	35
6.1	Concept de blockchain	35
6.2	Smart Contract de certification	35
6.3	Enregistrement des preuves	36
6.4	Interaction avec Ethereum	37
6.5	Stockage IPFS	38
6.6	Traçabilité et transparence	39
7	Axe Agricoins (AGC) et Datawarehouse	41
7.1	Présentation des Agricoins	41
7.2	Mécanisme d'émission	42
7.3	Paiement hybride (AGC + Mobile Money)	43
7.4	Gestion des transactions	44
7.5	Commerce et marketplace	44
8	Axe Sécurité et Cryptographie	46
8.1	Architecture de sécurité	46
8.2	Tableau récapitulatif des dix couches	47
8.3	Description détaillée des couches	48
8.3.1	C1 — Signatures ECDSA P-256 (Web Crypto API)	48
8.3.2	C2 — IndexedDB non extractable	48
8.3.3	C3 — Validation serveur (Cloud Functions)	49

8.3.4	C4 — Nonce anti-rejeu (UUID + timestamp)	49
8.3.5	C5 — Hash image SHA-256 (vérification IPFS serveur)	50
8.3.6	C6 — Ledger immuable (SHA-256 chaîné)	50
8.3.7	C7 — Chiffrement AES-GCM + PBKDF2 (310 000 itérations)	51
8.3.8	C8 — Escrow AGC (séquestre atomique)	52
8.3.9	C9 — Audit Trail immuable (journalisation write-only)	53
8.3.10	C10 — Firestore Security Rules (ownership + rate limiting)	53
8.4	Scénarios d'attaque et complémentarité des couches	54
8.5	Tests de sécurité automatisés	55
8.5.1	Architecture du tableau de bord	56
8.5.2	Jeu de tests implémentés	56
8.5.3	Résultats des tests	57
8.5.4	Extraits de code du tableau de bord	59
8.6	Bonnes pratiques de sécurité implémentées	61
8.7	Synthèse de l'axe Sécurité	61
	Conclusion Générale	63
	Bibliographie	64

Table des figures

1.1	Chiffres clés du secteur agricole sénégalais	2
1.2	Triple problématique et réponse intégrée de Jokko Agro	4
1.3	Les quatre axes fonctionnels de Jokko Agro	5
1.4	Acteurs de la plateforme Jokko Agro	6
2.1	Interfaces des principales solutions concurrentes	8
2.2	Analyse des lacunes des solutions existantes	9
3.1	Écran de connexion / inscription de Jokko Agro	11
3.2	Architecture en couches de Jokko Agro	12
3.3	Schéma simplifié des collections Firestore de Jokko Agro	13
3.4	Interaction des services d'intelligence artificielle	17
3.5	Flux de certification avec IPFS et Ethereum	19
3.6	Autres services externes intégrés dans Jokko Agro	20
3.7	Outils et technologies utilisés dans le développement de Jokko Agro	20
4.1	Architecture des services Angular de Jokko Agro	21
4.2	Vue d'ensemble des services Angular	22
4.3	Diagramme de classes UML — Architecture modulaire complète de Jokko Agro	23
4.4	Diagramme de séquence — Flux de certification blockchain	24
4.5	Diagramme de séquence — Interaction utilisateur avec l'IA	25
4.6	Diagramme de séquence simplifié — Paiement avec séquestre AGC	26
4.7	Flux de données entre les quatre axes technologiques	26
4.8	Couches de sécurité de Jokko Agro	28
5.1	Les quatre services de l'axe IA	30
5.2	Interface du chatbot agricole avec Claude API	31
5.3	Flux de traitement du chatbot agricole	31
5.4	Exemple de comparaison d'images anti-fraude avec TensorFlow.js	32

5.5	Validation GPS des points de certification	33
6.1	Structure d'une blockchain — principe de chaînage par hash	35
6.2	Interface de certification avec upload d'image	36
6.3	Popup MetaMask pour signature de la transaction	37
6.4	Flux complet de création d'une preuve de certification	37
6.5	Confirmation d'upload IPFS avec CID généré	38
6.6	Système de badges (Bronze/Silver/Gold) selon le niveau de certification ...	39
6.7	Détail d'un certificat blockchain avec QR code de vérification	39
6.8	QR code permettant la vérification instantanée de la certification	40
6.9	Les 7 checkpoints de certification et le système de badges	40
7.1	Portefeuille AGC	41
7.2	Interface d'achat des Agricoins (AGC)	42
7.3	Historique des transactions AGC	42
7.4	Interface de paiement avec Wave / Orange Money	43
7.5	Cycle de vie d'un paiement AGC avec escrow	44
7.6	Interface du marketplace avec produits certifiés	45
7.7	Machine à états d'une commande Jokko Agro	45
8.1	Architecture de sécurité en dix couches de Jokko Agro	46
8.2	Cycle de vie d'un paiement AGC avec escrow	52
8.3	Extrait des règles de sécurité Firestore	54
8.4	Extrait des règles de sécurité Firestore	54
8.5	Tableau de bord des tests de sécurité — Vue d'ensemble	56
8.6	Résultats des tests de sécurité sur la couche Ledger	58
8.7	Résultats détaillés des tests sur la signature ECDSA	58
8.8	Résultats des tests de sécurité sur le Chiffrement AEES	58
8.9	Résultats des tests de sécurité synthétisés	58

Liste des Abréviations

AGC	Agricoïn (monnaie numérique agricole)
API	Application Programming Interface
CID	Content Identifier (identifiant de contenu IPFS)
DW	Data Warehouse
ECDSA	Elliptic Curve Digital Signature Algorithm
IA	Intelligence Artificielle
IPFS	InterPlanetary File System
LLM	Large Language Model
PWA	Progressive Web App
RPC	Remote Procedure Call
SHA	Secure Hash Algorithm
TOTP	Time-based One-Time Password
2FA	Two-Factor Authentication
FCFA	Franç CFA
OM	Orange Money
UI	User Interface
UML	Unified Modeling Language

Introduction Générale

L'agriculture constitue le pilier économique de nombreux pays d'Afrique subsaharienne, employant plus de la moitié de la population active. Au Sénégal, elle représente environ **17% du PIB national** et fait vivre directement ou indirectement près de **70% de la population rurale**. Pourtant, malgré ce rôle central, les agriculteurs sénégalais demeurent confrontés à un triple défi structurel : un accès limité à l'expertise agronomique, une absence de traçabilité fiable des produits et des systèmes de paiement inadaptés aux réalités du terrain.

Face à cette réalité, le numérique s'impose comme un levier incontournable de transformation. Les récentes avancées en matière d'intelligence artificielle, de technologie blockchain et de paiement mobile ouvrent des perspectives nouvelles pour répondre concrètement à ces défis. C'est dans ce contexte que s'inscrit **Jokko Agro**, dont le nom — « *Jokko* », signifiant « *connexion* » en Wolof — traduit l'ambition fondatrice du projet : connecter les agriculteurs, les acheteurs et les experts au sein d'un écosystème numérique unifié, inclusif et sécurisé.

La plateforme repose sur quatre axes technologiques complémentaires : l'**Intelligence Artificielle** pour l'assistance agronomique bilingue français/Wolof, la **Blockchain Ethereum** pour la certification décentralisée des produits, les **Agricoins (AGC)** comme monnaie numérique interne couplée à un datawarehouse de pilotage, et un axe **Sécurité multicouche** garantissant l'intégrité de l'ensemble du système.

Ce mémoire présente l'architecture technique de Jokko Agro et détaille les choix de conception retenus. Il s'organise en huit chapitres : après une présentation générale du projet et une étude de l'existant, nous exposons les technologies utilisées, puis l'architecture globale du système. Les quatre chapitres suivants décrivent en profondeur chacun des axes technologiques. Le mémoire s'achève sur une conclusion qui dresse le bilan du travail accompli et ouvre sur les perspectives d'évolution de la plateforme.



Présentation Générale du Projet

Vue d'ensemble du chapitre

Ce chapitre introductif présente le cadre général du projet. Après une analyse du contexte agricole sénégalais et de ses contraintes structurelles, nous identifions la problématique centrale qui a motivé ce travail. Nous énonçons ensuite les objectifs poursuivis avant de décrire l'approche globale adoptée et de présenter les grandes lignes de la plateforme Jokko Agro.

1.1 Contexte et état des lieux du secteur agricole sénégalais



FIGURE 1.1 – Chiffres clés du secteur agricole sénégalais

L'agriculture occupe une place fondamentale dans l'économie du Sénégal. Elle emploie près de **70% de la population active** et contribue à environ **17% du PIB national**. Avec une population rurale dépassant **7,5 millions de personnes**, le secteur primaire conditionne directement les conditions de vie d'une majorité de ménages.

Les principales filières agricoles sont l'arachide, le mil, le sorgho, le maïs, le riz, la tomate, l'oignon et la mangue. Ces productions, majoritairement destinées à l'alimentation locale, représentent un enjeu majeur de souveraineté alimentaire.

Malgré ce potentiel avéré, le secteur agricole sénégalais se caractérise par des **contraintes structurelles profondes** qui limitent son développement et maintiennent les rendements bien en dessous des possibilités agronomiques des terres.

1.1.1 Caractéristiques structurelles des exploitations

L'agriculture sénégalaise est dominée par de petites exploitations familiales. Plusieurs traits structurels freinent leur modernisation :

- **Taille réduite des surfaces cultivées** : moins de 2 hectares en moyenne par exploitation ;
- **Faible mécanisation** et recours limité aux intrants modernes ;
- **Dépendance forte à la pluviométrie** et aux aléas climatiques ;
- **Accès restreint au crédit** et aux services financiers formels.

1.1.2 Une infrastructure numérique paradoxalement favorable

Paradoxalement, le Sénégal connaît une **pénétration rapide des technologies mobiles**. La couverture téléphonique dépasse **95% en zone urbaine** et **70% en milieu rural**. Le mobile money (Wave, Orange Money) comptabilise plus de **8 millions d'utilisateurs actifs** en 2024, preuve que les populations rurales sont prêtes à adopter des outils numériques adaptés.

Cependant, cette infrastructure numérique demeure sous-exploitée dans le secteur agricole. Les agriculteurs restent largement exclus des circuits formels de conseil, de certification et de paiement, malgré l'existence d'un potentiel d'innovation technologique important.

1.2 Problématique et question centrale

À partir de ce diagnostic contrasté, une problématique centrale émerge. Elle peut être formulée comme suit :

Comment permettre aux agriculteurs sénégalais — souvent faiblement bancarisés, disposant d'une formation limitée et vivant dans des zones peu connectées — de bénéficier des avancées technologiques pour améliorer leurs pratiques agricoles, sécuriser la traçabilité de leurs productions et s'insérer durablement dans des circuits économiques formels et équitables ?

Cette question centrale se décompose en trois sous-problèmes interdépendants :

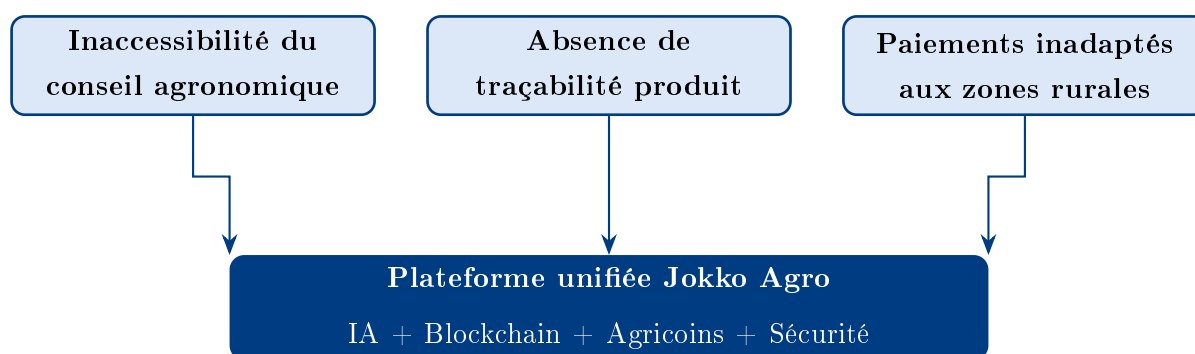


FIGURE 1.2 – Triple problématique et réponse intégrée de Jokko Agro

Ces trois sous-problèmes sont structurellement liés : un agriculteur sans conseil ne peut améliorer la qualité de ses produits ; sans traçabilité, cette qualité ne trouve pas de valorisation marchande ; sans système de paiement adapté, la transaction commerciale ne peut être sécurisée. C'est pourquoi une réponse intégrée, adressant simultanément ces trois dimensions, est nécessaire.

1.3 Objectifs du projet

Face à cette problématique, Jokko Agro s'est fixé un objectif général décliné en cinq objectifs spécifiques.

1.3.1 Objectif général

L'**objectif général** de ce travail est de concevoir et développer une plateforme numérique agricole unifiée, adaptée au contexte sénégalais, permettant aux agriculteurs d'accéder à des conseils personnalisés, de certifier leurs produits et de réaliser des transactions sécurisées dans un environnement de confiance.

1.3.2 Objectifs spécifiques

Objectifs spécifiques

- OS1.** Déployer un **assistant agricole intelligent bilingue** (français/Wolof) accessible par texte et par voix pour démocratiser l'accès au conseil agronomique.
- OS2.** Développer un **système de certification basé sur la blockchain** enregistrant de façon immuable les étapes du cycle de production, du semis à la commercialisation.
- OS3.** Créer une **monnaie numérique agricole** (Agricoins ou AGC), convertible en FCFA via Wave et Orange Money, pour faciliter les micro-transactions en milieu rural.
- OS4.** Garantir la **sécurité cryptographique** de l'ensemble du système en mettant en œuvre des mécanismes robustes.

1.4 Approche globale : une réponse intégrée en quatre axes

Pour répondre aux objectifs fixés, nous avons adopté une approche modulaire organisée autour de **quatre axes fonctionnels** complémentaires et interconnectés. Cette architecture permet d'adresser chaque sous-problème avec la technologie la plus adaptée, tout en maintenant une cohérence d'ensemble.

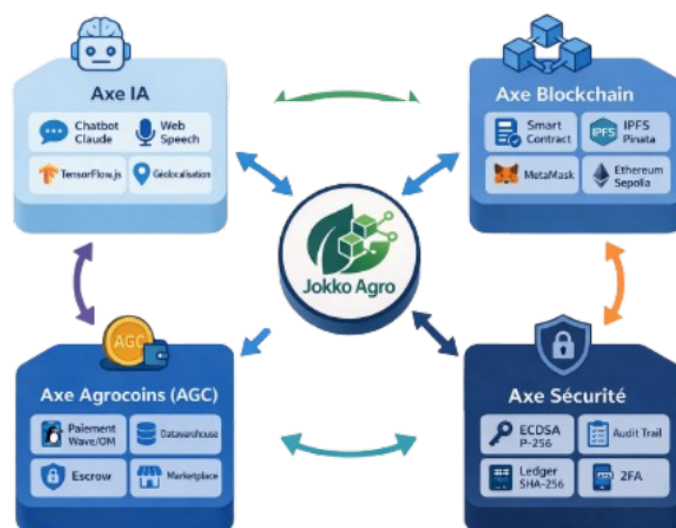


FIGURE 1.3 – Les quatre axes fonctionnels de Jokko Agro

Chaque axe répond à un besoin spécifique identifié dans la problématique :

- **Axe 1 — Intelligence Artificielle** : démocratiser l'accès au conseil agronomique

via un assistant accessible en français et en Wolof, par texte ou par voix ;

- **Axe 2 — Blockchain et Certification** : instaurer un système de confiance grâce à une traçabilité immuable des produits, du semis à la vente ;
- **Axe 3 — Agricoins et Datawarehouse** : faciliter les transactions financières en milieu rural et fournir des outils d'analyse de performance ;
- **Axe 4 — Sécurité multicouche** : protéger l'intégrité des données et des transactions des utilisateurs.

1.5 Présentation synthétique de la plateforme

Jokko Agro est une application web progressive (PWA) dont le nom — « *Jokko* », signifiant « *connexion* » en Wolof — traduit l'ambition fondatrice : créer un pont numérique entre les agriculteurs sénégalais et les outils technologiques modernes.

1.5.1 Acteurs de la plateforme

La plateforme met en relation deux types d'acteurs aux intérêts complémentaires :

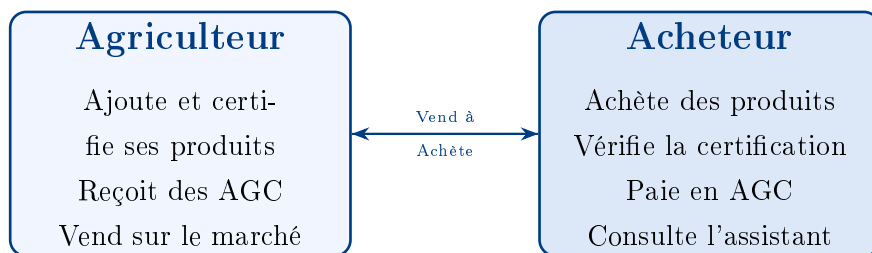


FIGURE 1.4 – Acteurs de la plateforme Jokko Agro

1.5.2 Organisation du mémoire

La suite de ce mémoire est organisée comme suit :

- **Chapitre 2** : État de l'art — analyse des solutions existantes et positionnement de Jokko Agro ;
- **Chapitre 3** : Technologies utilisées — présentation détaillée des briques technologiques retenues ;
- **Chapitre 4** : Architecture globale — description de l'architecture technique et des interactions entre composants ;
- **Chapitres 5 à 8** : Présentation détaillée des quatre axes (IA, Blockchain, Agricoins, Sécurité) ;
- **Conclusion** : bilan du travail réalisé et perspectives d'évolution.

« Jokko » : connecter l'agriculture sénégalaise aux technologies de demain.

État de l'art

Vue d'ensemble du chapitre

Ce chapitre dresse un panorama des solutions numériques agricoles existantes en Afrique subsaharienne. Après avoir exposé notre méthodologie d'analyse, nous présentons les plateformes les plus représentatives, identifions leurs limites communes, et positionnons Jokko Agro comme une réponse intégrée aux lacunes constatées.

2.1 Méthodologie d'analyse

L'analyse comparative présentée dans ce chapitre repose sur une revue systématique des solutions numériques agricoles opérant en Afrique subsaharienne. Notre méthodologie comprend :

- Une recherche bibliographique sur les bases **Scopus**, **Google Scholar** et **Web of Science** avec les mots-clés : *"agricultural digital platform"*, *"blockchain agriculture Africa"*, *"agritech Senegal"* ;
- Une analyse des **rapports sectoriels** (GSMA, FAO, Agence Française de Développement) publiés entre 2020 et 2024 ;
- L'étude de **5 plateformes représentatives** sélectionnées selon trois critères : présence en Afrique de l'Ouest, fonctionnalités innovantes, données disponibles sur l'impact réel.

2.2 Analyse des solutions existantes

Les initiatives numériques dans le secteur agricole ouest-africain se répartissent en trois catégories fonctionnelles : (1) les plateformes de mise en relation marchande, (2) les applications de conseil agricole, (3) les solutions de traçabilité et de financement. Le tableau ?? synthétise les principales plateformes.

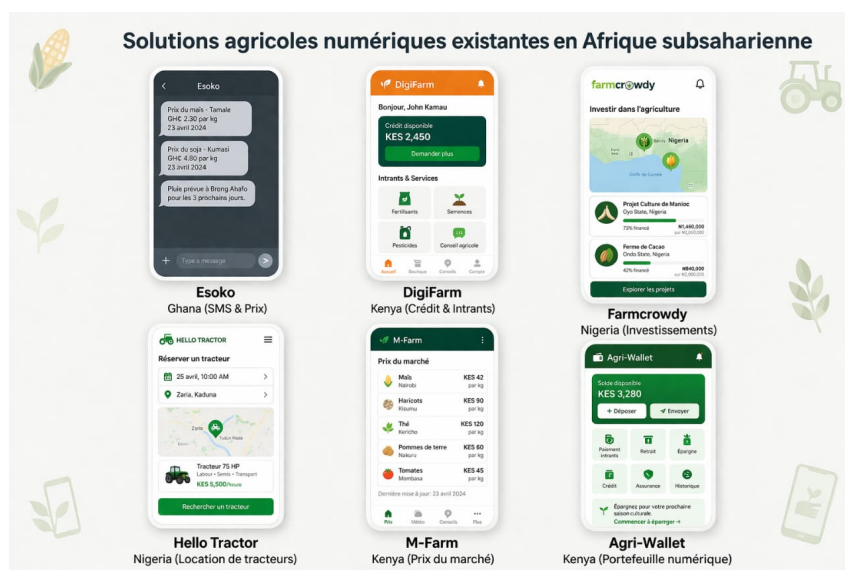


FIGURE 2.1 – Interfaces des principales solutions concurrentes

2.2.1 Analyse détaillée des plateformes

Solution	Pays	Lancement	Utilisateurs	Limite principale
Esoko	Ghana / Mali	2005	500 000+	Pas de traçabilité ni d'IA
Farmcrowdy	Nigeria	2016	25 000+	Pas de certification blockchain
Hello Tractor	Nigeria	2014	100 000+	Hors chaîne de valeur
M-Farm	Kenya	2012	50 000+	Langue anglaise uniquement
Agri-Wallet	Kenya	2019	30 000+	Pas d'IA ni de certification
DigiFarm	Kenya	2017	1 M+	Non adapté au Sénégal / Wolof

Sources : Rapports GSMA 2023, sites officiels des plateformes

2.3 Lacunes des systèmes existants

L'analyse comparative met en évidence quatre lacunes structurelles, que nous avons formalisées comme autant de verrous à lever :

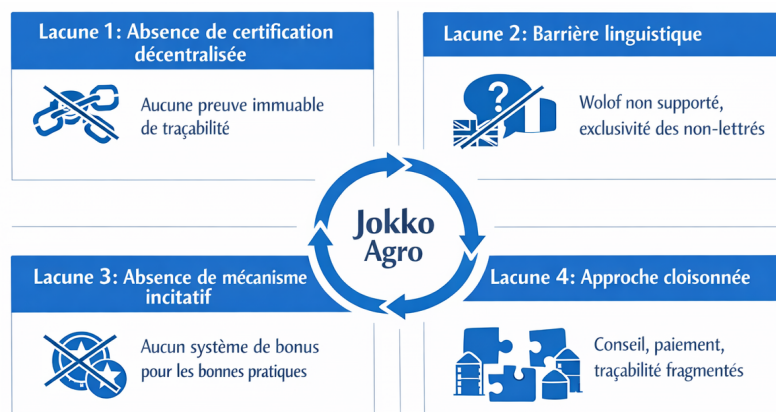


FIGURE 2.2 – Analyse des lacunes des solutions existantes

Synthèse des lacunes identifiées

- L1. Absence de certification décentralisée** : aucune solution ne s'appuie sur la blockchain pour garantir l'immuabilité des preuves de production. *Conséquence* : impossibilité de prouver l'origine et la qualité auprès des acheteurs exigeants (export, grande distribution).
- L2. Barrière linguistique et technique** : les applications existantes fonctionnent en anglais ou en français standard, excluant les populations rurales non scolarisées ou ne maîtrisant pas ces langues. *Conséquence* : fracture numérique doublée d'une fracture linguistique.
- L3. Absence de mécanisme incitatif** : aucune solution ne propose un système de récompense monétaire (reward token) lié aux bonnes pratiques agricoles ou à la qualité de la certification. *Conséquence* : faible motivation des agriculteurs à s'approprier durablement les outils numériques.
- L4. Approche cloisonnée** : les solutions traitent un seul problème (conseil, paiement ou traçabilité) sans les intégrer dans un écosystème unifié. *Conséquence* : multiplicité des applications, lassitude des utilisateurs, données fragmentées.

2.4 Positionnement stratégique de Jokko Agro

Face à ces lacunes, Jokko Agro adopte un positionnement original qui combine deux dimensions jusqu'ici disjointes : l'innovation technologique (IA, blockchain) et l'inclusivité linguistique (Wolof, interface vocale).

2.5 Valeur ajoutée et innovations

Jokko Agro se distingue par une combinaison unique de quatre innovations qui répondent directement aux lacunes identifiées :

Innovation 1 — IA multilingue : Premier chatbot agricole bilingue français/Wolof basé sur un LLM (Claude API), avec reconnaissance vocale adaptée aux langues locales. *Répond à L2.*

Innovation 2 — Blockchain certifiante : Système de certification décentralisé avec preuves photographiques géolocalisées, enregistrées de façon immuable sur Ethereum Sepolia et IPFS. *Répond à L1.*

Innovation 3 — Monnaie incitative (AGC) : Token agricole (1 AGC = 100 FCFA) convertible en mobile money, avec système de récompense pour les bonnes pratiques et la certification. *Répond à L3.*

Innovation 4 — Écosystème intégré : Architecture unifiée combinant conseil, traçabilité et paiement dans une seule interface accessible par mobile. *Répond à L4.*

2.6 Synthèse et conclusion du chapitre

L'analyse comparative menée dans ce chapitre aboutit à trois constats principaux :

1. Les solutions existantes, bien que nombreuses, souffrent d'une **approche sectorielle** qui limite leur impact systémique ;
2. Aucune plateforme ne combine actuellement **IA multilingue, blockchain certifiante et monnaie incitative** dans une architecture unifiée ;
3. Le contexte sénégalais, avec sa spécificité linguistique (Wolof) et son écosystème de mobile money mature, constitue un **terrain fertile** pour une solution innovante.

C'est sur la base de ces constats que nous avons défini l'architecture de Jokko Agro, détaillée dans les chapitres suivants.

« L'innovation ne consiste pas à ajouter de la technologie, mais à la rendre accessible à ceux qui en ont le plus besoin. »

Technologies et Outils Utilisés

Vue d'ensemble du chapitre

Ce chapitre décrit les technologies et outils retenus pour la conception de Jokko Agro. Pour chaque technologie, nous expliquons le choix effectué, ses caractéristiques techniques et son rôle précis dans la plateforme. L'accent est mis sur les technologies d'intelligence artificielle (TensorFlow.js, Claude API, Web Speech API) qui constituent le cœur de l'axe IA.

3.1 Architecture technique globale

L'architecture technique de Jokko Agro est organisée en quatre couches indépendantes mais interconnectées, chacune jouant un rôle précis dans le cycle de vie des données.

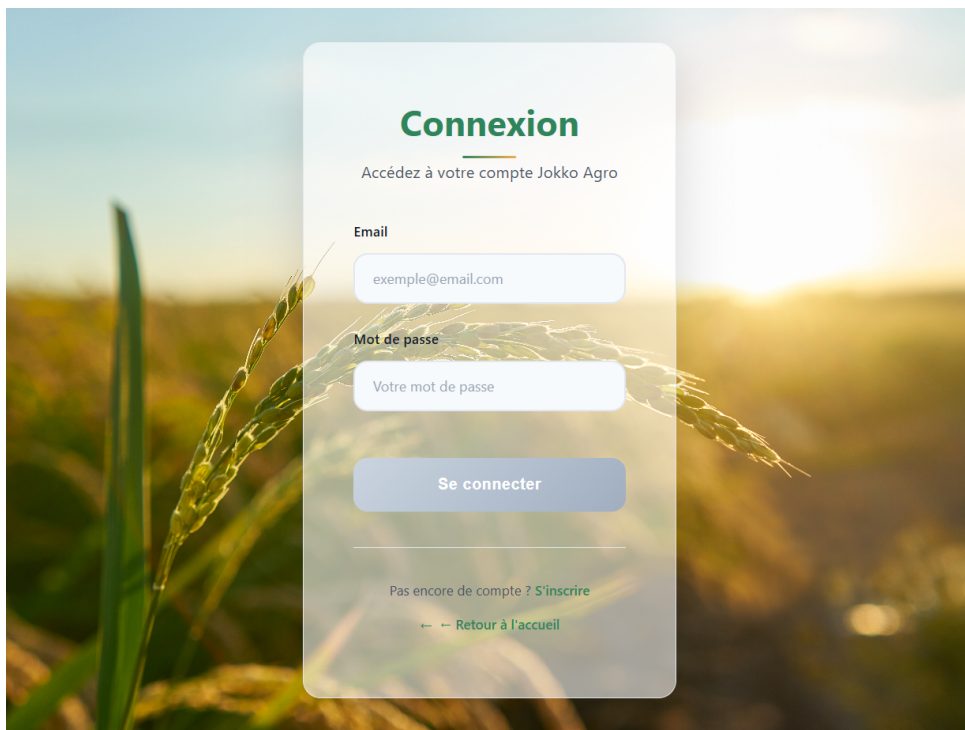


FIGURE 3.1 – Écran de connexion / inscription de Jokko Agro

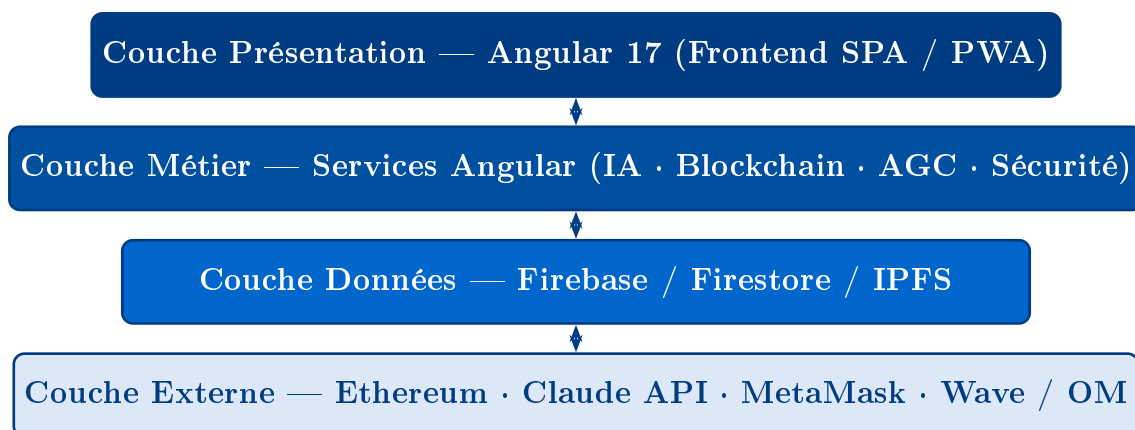


FIGURE 3.2 – Architecture en couches de Jokko Agro

3.2 Frontend : Angular 17



Angular 17 est le framework JavaScript retenu pour le développement de l'interface utilisateur. Il s'agit d'un framework front-end développé par Google, basé sur TypeScript, qui permet la construction d'applications web monopages (SPA) robustes et maintenables.

Pourquoi Angular 17 ?

- **Standalone components** : réduction du boilerplate, modules allégés.
- **RxJS intégré** : gestion réactive des états (wallet, solde AGC) via BehaviorSubject.
- **TypeScript natif** : typage fort, détection d'erreurs à la compilation.
- **PWA support** : application installable sur mobile sans store.
- **Lazy loading** : chargement à la demande des modules lourds (TensorFlow.js).

3.3 Backend : Firebase / Firestore



Firebase (Google) est utilisé comme plateforme backend-as-a-service. Il fournit l'authentification, la base de données en temps réel (Firestore), les Cloud Functions et l'hébergement.

Fonctionnalités Firebase utilisées

- **Authentication** : gestion des utilisateurs (email/mot de passe, Google)
- **Firestore** : base de données NoSQL temps réel
- **Cloud Functions** : logique métier serverless (validation AGC)
- **Security Rules** : contrôle d'accès granulaire

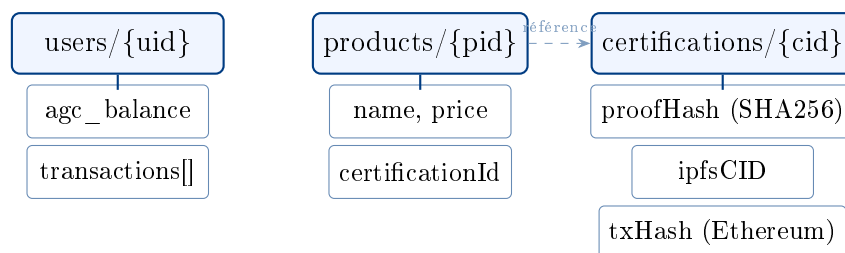


FIGURE 3.3 – Schéma simplifié des collections Firestore de Jokko Agro

3.4 Intelligence Artificielle

L'axe IA de Jokko Agro repose sur trois briques technologiques complémentaires, chacune sélectionnée pour son adéquation avec les contraintes du contexte sénégalais (accessibilité mobile, langues locales, traitement en temps réel).

3.4.1 TensorFlow.js — Vision par ordinateur côté client



TensorFlow.js est la bibliothèque JavaScript de Google permettant d'exécuter des modèles d'apprentissage automatique directement dans le navigateur. Dans Jokko Agro, elle est utilisée pour la **comparaison d'images anti-fraude** lors du processus de certification.

Pourquoi TensorFlow.js plutôt que Python ?

- **Côté client** : pas d'envoi d'images sensibles vers un serveur, préservation de la confidentialité des agriculteurs
- **Hors ligne** : les comparaisons d'images fonctionnent sans connexion internet (mode dégradé)
- **Latence nulle** : pas de temps d'attente serveur, retour immédiat
- **Coût réduit** : pas d'infrastructure GPU à provisionner

Le service `image-comparison.service.ts` implémente cinq métriques de similarité qui sont combinées pour détecter les fraudes :

Métriques de comparaison d'images avec TensorFlow.js

1. **Mean Squared Error (MSE)** : différence quadratique moyenne entre pixels
2. **Cosine Similarity** : similarité angulaire des vecteurs d'image
3. **Histogram Correlation** : corrélation des histogrammes de couleurs
4. **SSIM (Structural Similarity)** : similarité structurelle
5. **Edge Detection Consistency** : cohérence des contours (Canny)

Extrait du service de comparaison d'images

```

1  // image-comparison.service.ts
2  import * as tf from 'tensorflow/tfjs';
3
4  export class ImageComparisonService {
5    async compareImages(img1: HTMLImageElement, img2: HTMLImageElement) {
6      const tensor1 = tf.browser.fromPixels(img1).toFloat();
7      const tensor2 = tf.browser.fromPixels(img2).toFloat();
8
9      // 1. MSE (Mean Squared Error)
10     const mse = tf.mean(tf.square(tf.sub(tensor1, tensor2))).dataSync()[0];
11
12     // 2. Cosine similarity
13     const norm1 = tf.norm(tensor1);
14     const norm2 = tf.norm(tensor2);
15     const dot = tf.sum(tf.mul(tensor1, tensor2));
16     const cosine = dot.div(norm1.mul(norm2)).dataSync()[0];
17
18     // 3. Histogram comparison (RGB channels)
19     const histScore = this.compareHistograms(tensor1, tensor2);
20
21     // Score global (ponderation)
22     const fraudScore = (mse * 0.3) + ((1 - cosine) * 0.4) + (histScore * 0.3);
23
24     return {
25       isFraudulent: fraudScore > 0.85,
26       scores: { mse, cosine, histScore, fraudScore },
27       recommendation: fraudScore > 0.85 ? 'Rejeter' : 'Accepter'
28     };
29   }
30 }

```

3.4.2 Claude API (Anthropic) — Chatbot agricole multilingue



Claude est le modèle de langage (LLM) développé par Anthropic, retenu pour ses performances en français et sa capacité à suivre des instructions précises. L'API Claude est utilisée pour le chatbot agronomique, accessible en français et en Wolof.

Pourquoi Claude plutôt que GPT-4 ?

- **Contexte long** : fenêtre de 200k tokens, idéale pour les conversations agricoles détaillées
- **Performance en français** : meilleure compréhension du français oral et des termes agronomiques locaux
- **Sécurité intégrée** : mécanismes de refus natifs pour les requêtes hors périmètre agricole
- **Coût** : tarifs plus compétitifs que GPT-4 Turbo

Fonctionnalités Claude utilisées

- **Claude 3.5 Sonnet** : modèle principal pour le chatbot
- **System prompts** : contexte agricole sénégalais injecté à chaque requête
- **Tool use** : recherche dans la base de connaissances agronomique
- **Streaming** : réponse en temps réel (effet "typing")

Extrait du service Claude

```

1 // claude-chat.service.ts
2 export class ClaudeChatService {
3   private readonly systemPrompt = `
4     Tu es Jokko Agro, assistant agricole expert pour le Senegal.
5     Tu parles francais et wolof.
6     Connaissances: cultures (arachide, mil, mais, riz, tomate, oignon),
7     techniques culturales, gestion de l'eau, lutte integree.
8     Reponds avec des conseils pratiques, adaptes aux petits exploitants.
9   `;
10
11   async askQuestion(question: string, language: 'fr' | 'wo'): Promise<string> {
12     const response = await fetch('https://api.anthropic.com/v1/messages', {
13       method: 'POST',
14       headers: {
15         'x-api-key': this.apiKey,
16         'anthropic-version': '2023-06-01',
17         'content-type': 'application/json'
18       },
19       body: JSON.stringify({
20         model: 'claude-3-5-sonnet-20241022',
21         max_tokens: 1024,
22         system: this.systemPrompt,
23         messages: [{ role: 'user', content: question }]
24       })
25     });
26     return (await response.json()).content[0].text;
27   }
28 }

```

3.4.3 Claude Vision — Analyse d'images de cultures

Au-delà du chatbot textuel, Jokko Agro utilise **Claude Vision** (modèle multimodal) pour analyser les photos de cultures envoyées par les agriculteurs. Cette fonctionnalité permet

la détection de maladies, l'estimation de rendement et la validation des certifications.

Cas d'usage de Claude Vision

- **Détection de maladies** : analyse des feuilles (mildiou, rouille, chenilles)
- **Estimation de maturité** : détermination du stade optimal de récolte
- **Validation certification** : vérification que la photo correspond bien à la culture déclarée
- **Recommandations** : conseils agronomiques basés sur l'observation visuelle

Extrait du service Claude Vision

```

1 // claude-vision.service.ts
2 export class ClaudeVisionService {
3   async analyzePlantImage(imageBase64: string, cropType: string): Promise<PlantAnalysis> {
4     const response = await fetch('https://api.anthropic.com/v1/messages', {
5       method: 'POST',
6       headers: { 'x-api-key': this.apiKey },
7       body: JSON.stringify({
8         model: 'claude-3-5-sonnet-20241022',
9         max_tokens: 1024,
10        messages: [{
11          role: 'user',
12          content: [
13            { type: 'text', text: 'Analyse cette culture de ${cropType}.
14                               Detecte les maladies, estime la maturite,
15                               donne des recommandations.' },
16            { type: 'image', source: { type: 'base64', media_type: 'image/jpeg', data: imageBase64 } }
17          ]
18        }]
19      });
20     return this.parseAnalysis(await response.json());
21   }
22 }
23 
```

3.4.4 Web Speech API — Reconnaissance vocale pour les langues locales



Web Speech API est une API native des navigateurs modernes qui permet la reconnaissance vocale côté client. Dans Jokko Agro, elle est utilisée pour permettre aux agriculteurs non lecteurs d'interagir avec le chatbot en Wolof ou en français.

Pourquoi Web Speech API ?

- **Natif navigateur** : aucune bibliothèque externe, léger
- **Hors ligne** : fonctionne sans connexion (mode dégradé)
- **Reconnaissance continue** : sessions de dialogue fluides
- **Gratuit** : pas de coût API par utilisateur

La reconnaissance vocale est enrichie par un post-traitement spécifique au Wolof :

Pipeline de reconnaissance vocale

1. Capture audio via `webkitSpeechRecognition` et Transcription brute en Français
2. Post-traitement phonologique pour adapter au Wolof :
 - Substitution phonétique (`/r/` → `/l/` en contexte)
 - Correction des termes et élection des noms en Wolof (`maaro` = mil)
3. Envoi du texte normalisé au chatbot Claude

Extrait du service de reconnaissance vocale

```

1  // speech-recognition.service.ts
2  export class SpeechRecognitionService {
3    private recognition: SpeechRecognition;
4
5    constructor() {
6      this.recognition = new webkitSpeechRecognition();
7      this.recognition.lang = 'fr-FR';
8      this.recognition.interimResults = true;
9      this.recognition.continuous = true;
10   }
11   startListening(): Observable<string> {
12     return new Observable(observer => {
13       this.recognition.onresult = (event) => {
14         let rawText = event.results[event.results.length-1][0].transcript;
15         let normalized = this.postProcessWolof(rawText);
16         observer.next(normalized);
17       };
18       this.recognition.start();
19     });
20   }
21   private postProcessWolof(text: string): string {
22     const corrections: { [key: string]: string } = {
23       'milo': 'mil', 'maaro': 'mil', 'souna': 'mais', 'tomat': 'tomate', 'gname': 'oignon'
24     };
25     let normalized = text.toLowerCase();
26     for (const [wolof, french] of Object.entries(corrections)) {
27       normalized = normalized.replace(new RegExp(wolof, 'g'), french);
28     }
29     return normalized;
30   }
31 }

```

3.4.5 Schéma d'interaction des services IA

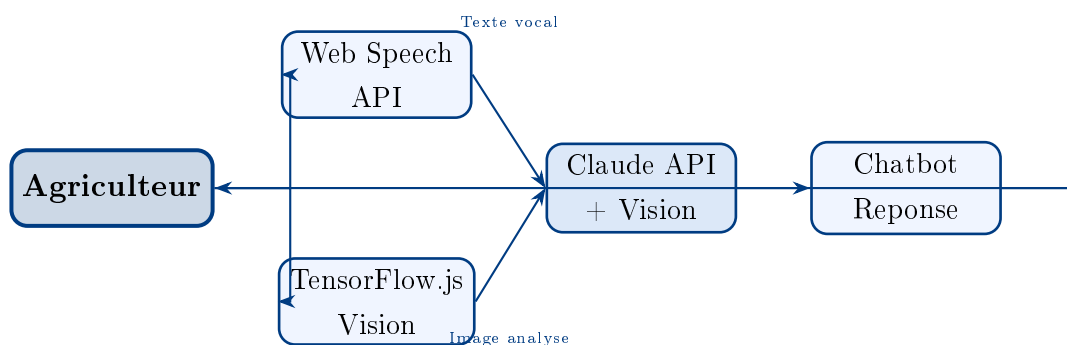


FIGURE 3.4 – Interaction des services d'intelligence artificielle

3.4.6 Synthèse des technologies IA

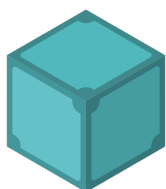
Récapitulatif des technologies IA

Technologie	Usage	Exécution	Avantage
TensorFlow.js	Vision fraude	anti- Navigateur client	Confidentialite, zero latence
Claude API	Chatbot nomique	agro- API externe	Multilingue, contexte long
Claude Vision	Analyse cultures	de API externe	Diagnostic multimodal
Web API	Speech Reconnaissance vocale	Navigateur client	Gratuit, hors ligne

3.5 Blockchain : Ethereum & Smart Contracts



Ethereum est la plateforme blockchain retenue pour la certification des produits agricoles. Le réseau **Sepolia Testnet** (chainId : 11155111) est utilisé en développement. Le smart contract CertificationRegistry est rédigé en **Solidity ^0.8.19**.



IPFS (InterPlanetary File System) est utilisé pour le stockage décentralisé des photos de certification.



MetaMask est l'extension de navigateur permettant aux utilisateurs de gérer leur wallet Ethereum et de signer les transactions de certification.

Caractéristiques Ethereum

- **Réseau** : Sepolia Testnet (chainId : 11155111)
- **Smart Contract** : Solidity ^0.8.19
- **Transactions** : signatures ECDSA

3.5.1 Flux de certification blockchain

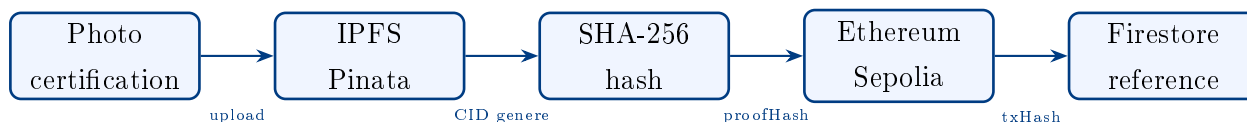


FIGURE 3.5 – Flux de certification avec IPFS et Ethereum

Extrait du Smart Contract

```

1  pragma solidity ^0.8.19;
2  contract CertificationRegistry {
3      struct CertificationProof {
4          string productId;
5          string proofHash; // Hash SHA-256 de la preuve
6          string ipfsCID; // Identifiant IPFS de la photo
7          string step; // INIT | FOLLOW_UP | HARVEST | CHECKPOINT
8          uint256 timestamp; // Horodatage blockchain (immuable)
9          address certifiedBy; // Adresse Ethereum du certificateur
10     }
11     event ProofRegistered(
12         string indexed productId,
13         string proofHash,
14         string ipfsCID,
15         uint256 timestamp,
16         address indexed certifier
17     );
18     mapping(string => CertificationProof) private proofs;
19     function registerProof(string memory _productId,
20         string memory _proofHash,
21         string memory _ipfsCID,
22         string memory _step) public {
23         proofs[_proofHash] = CertificationProof(
24             _productId, _proofHash, _ipfsCID,
25             _step, block.timestamp, msg.sender
26         );
27         emit ProofRegistered(_productId, _proofHash,
28             _ipfsCID, block.timestamp, msg.sender);
29     }
30 }

```

3.6 Services externes

3.6.1 Alchemy — Provider RPC

Alchemy est le fournisseur RPC (Remote Procedure Call) utilisé pour interroger la blockchain Ethereum sans nœud local. Il offre une API JSON-RPC fiable et des analytics de transactions.

3.6.2 Wave et Orange Money — Mobile money

Wave et Orange Money sont les plateformes de mobile money intégrées pour l'achat d'Agricoins (AGC). Leur adoption massive au Sénégal (plus de 8M d'utilisateurs combinés) garantit l'accessibilité du système de paiement.



FIGURE 3.6 – Autres services externes intégrés dans Jokko Agro

3.7 Outils de développement

Stack de développement

- **IDE** : Visual Studio Code avec extensions Angular, Python, Solidity
- **Versioning** : Git / GitHub (branches feature, develop, main)
- **Déploiement smart contract** : Remix IDE + Hardhat
- **Test blockchain** : Sepolia Faucet (ETH de test)
- **Monitoring** : Firebase Console + Etherscan Sepolia
- **Cryptographie** : WebCrypto API (natif navigateur) + CryptoJS
- **IA / ML** : TensorFlow.js, Claude API (Anthropic)



FIGURE 3.7 – Outils et technologies utilisés dans le développement de Jokko Agro

« La technologie la plus avancée est celle qui disparaît au service de l'utilisateur. »

Conception et Architecture du Système

Vue d'ensemble du chapitre

Ce chapitre présente l'architecture complète de Jokko Agro à travers ses diagrammes UML, ses flux de données inter-modules et ses principes de conception. Il constitue le blueprint technique de la plateforme, détaillant l'organisation des services, les interactions entre les composants et les mécanismes de scalabilité.

4.1 Architecture globale du système

L'architecture de Jokko Agro suit le modèle **SPA mono-repo Angular** avec séparation stricte en modules services. L'intégralité de la logique métier est implémentée dans des services Angular injectables, ce qui permet les tests unitaires et la réutilisabilité des composants.

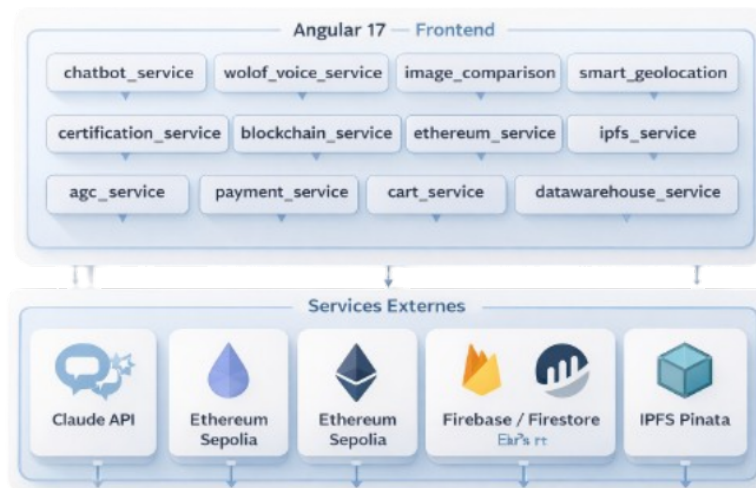


FIGURE 4.1 – Architecture des services Angular de Jokko Agro

4.2 Architecture modulaire

La plateforme est organisée en **quatre modules fonctionnels** correspondant aux quatre axes technologiques. Chaque module expose ses services via l'injection de dépendances Angular, garantissant un couplage faible entre les modules.

Principes de l'architecture modulaire

- **Séparation des responsabilités** : chaque service a une responsabilité unique (Single Responsibility Principle).
- **Injection de dépendances** : les services s'injectent mutuellement via le conteneur IoC d'Angular, sans couplage direct.
- **Observables RxJS** : les états partagés (solde AGC, état wallet) sont exposés via BehaviorSubject pour une réactivité globale.
- **Lazy loading** : les modules lourds (TensorFlow.js) sont chargés à la demande pour optimiser le temps de démarrage.

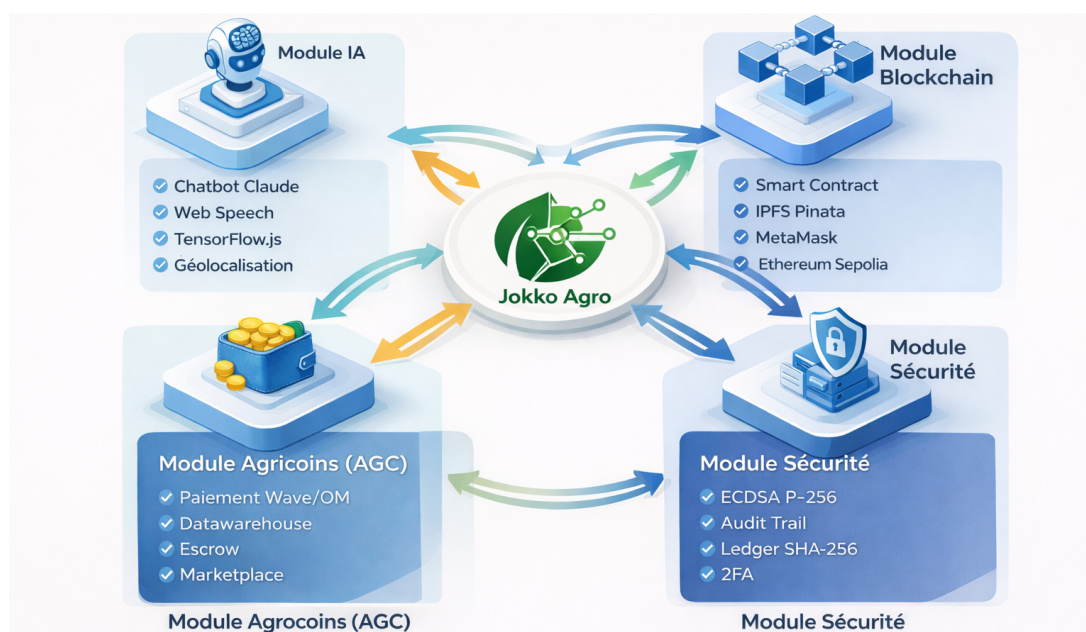


FIGURE 4.2 – Vue d'ensemble des services Angular

4.3 Diagramme de classes UML

Le diagramme de classes présente la structure des entités principales et leurs relations. Les quatre modules métier sont représentés avec leurs attributs et méthodes clés.

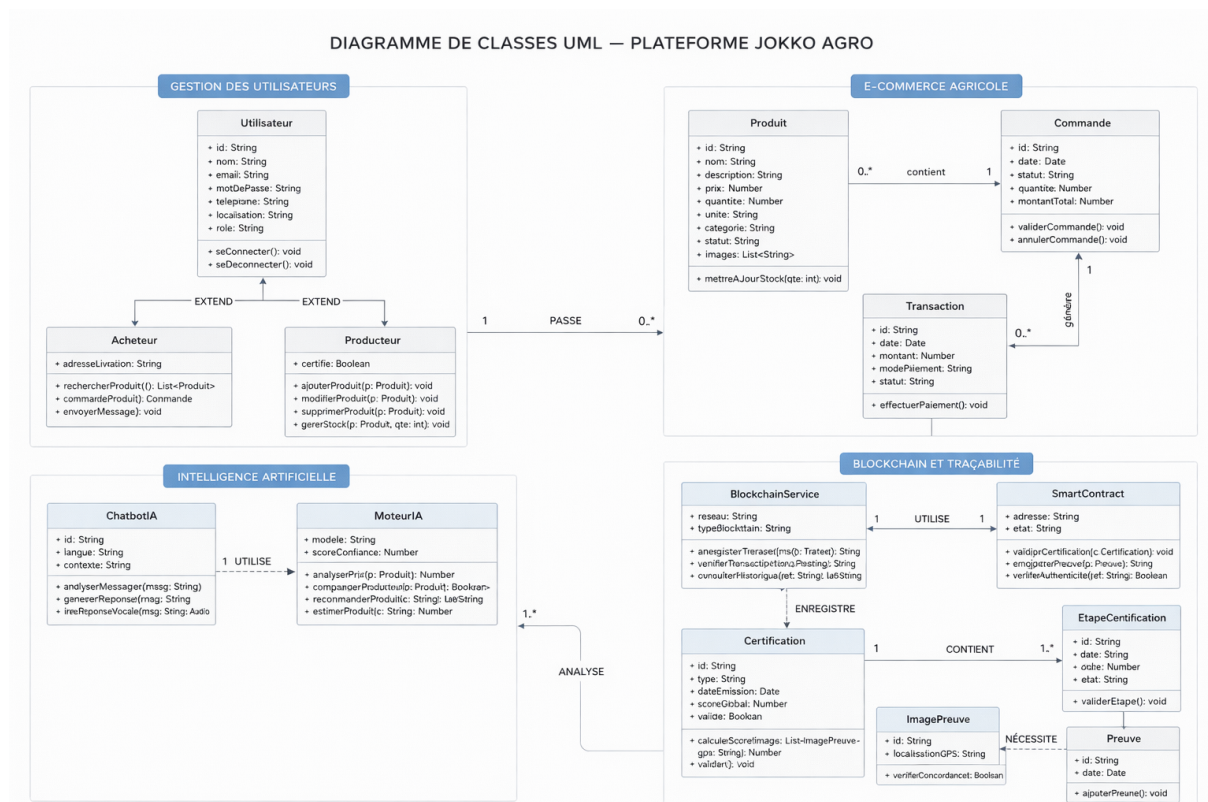


FIGURE 4.3 – Diagramme de classes UML — Architecture modulaire complète de Jokko Agro

4.3.1 Description des entités principales

Entités principales du modèle de données

- **User** : représentant un agriculteur ou un acheteur. Attributs : uid, email, role, walletAddress, agcBalance.
- **Product** : produit agricole mis en vente. Attributs : productId, name, price, certificationId, farmerId.
- **Certification** : preuve de traçabilité. Attributs : certId, productId, proofHash, ipfsCID, txHash, timestamp, checkpoint (7 étapes).
- **Transaction** : échange d'Agricoins. Attributs : txId, senderId, receiverId, amount, status, signature.
- **AuditLog** : trace de toutes les actions sensibles. Attributs : logId, userId, action, timestamp, ipfsHash (preuve d'intégrité).

4.4 Diagrammes de séquence

4.4.1 Séquence Blockchain

Le diagramme suivant illustre le flux complet de certification d'un produit agricole, depuis la prise de photo par l'agriculteur jusqu'à l'enregistrement on-chain.

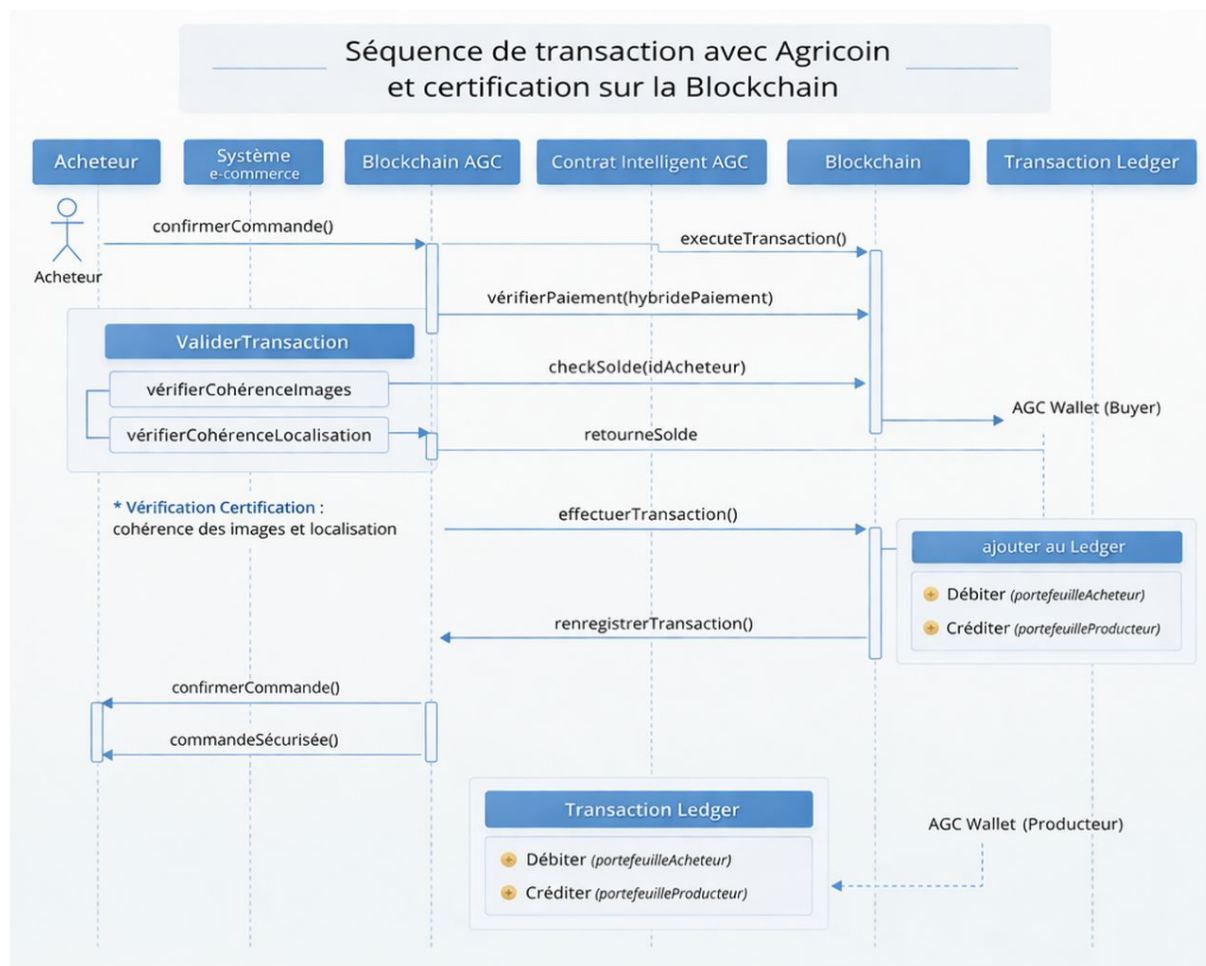


FIGURE 4.4 – Diagramme de séquence — Flux de certification blockchain

Le flux se déroule en **six étapes** :

1. **Prise de photo + validation GPS** : l'agriculteur capture une photo de sa culture, le service `smart_geolocation` vérifie la cohérence des coordonnées GPS.
2. **Calcul du hash SHA-256** : la photo et les métadonnées sont hachées pour créer une empreinte unique.
3. **Upload IPFS via Pinata** : la photo est stockée de manière décentralisée, un CID est généré.
4. **Construction de l'objet canonique** : les preuves sont regroupées dans une structure JSON signée.
5. **Transaction Ethereum signée MetaMask** : l'utilisateur signe la transaction avec son wallet, enregistrement sur Sepolia.

6. **Enregistrement dans Firestore** : les références (txHash, CID) sont sauvegardées pour une consultation rapide.

4.4.2 Séquence IA

Ce diagramme décrit l'interaction entre l'utilisateur et le chatbot, en montrant le traitement de la voix Wolof, l'appel à l'API Claude et le retour de la réponse.

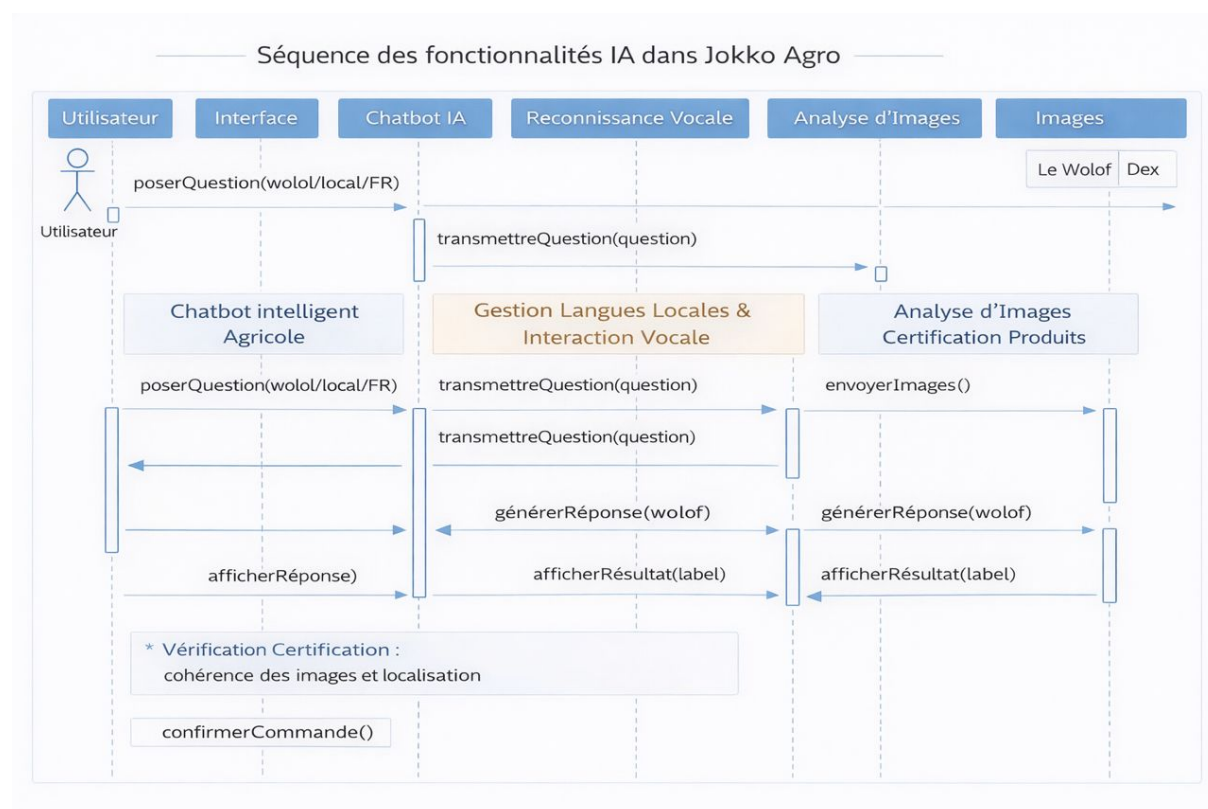


FIGURE 4.5 – Diagramme de séquence — Interaction utilisateur avec l'IA

Le traitement vocal suit un pipeline spécifique :

1. **Capture audio** : l'utilisateur parle en Wolof ou en français.
2. **Transcription Web Speech API** : reconnaissance vocale native.
3. **Post-traitement Wolof** : correction phonétique et substitution des termes agricoles locaux.
4. **Appel Claude API** : envoi du texte normalisé avec contexte agronomique.
5. **Génération de la réponse** : Claude produit un conseil personnalisé.
6. **Synthèse vocale (TTS)** : retour audio en langue locale.

4.4.3 Séquence AGC et Paiement

Ce diagramme illustre le flux d'achat d'un produit certifié avec paiement en Agricoins, incluant le mécanisme de séquestre (escrow).



FIGURE 4.6 – Diagramme de séquence simplifié — Paiement avec séquestre AGC

4.5 Flux de données

Les quatre axes de Jokko Agro échangent des données à des points clés. Ces interactions sont essentielles à la cohérence du système :

Interactions inter-axes

- **IA** ↔ **Blockchain** : le service `image_comparison_service` valide la photo *avant* que `blockchain_service` l'enregistre sur IPFS. Les coordonnées GPS validées sont incluses dans le hash de preuve canonique.
- **Blockchain** ↔ **Sécurité** : chaque enregistrement on-chain est précédé d'une signature ECDSA et journalisé dans l'Audit Trail. Le Ledger SHA-256 trace tous les états de certification.
- **AGC** ↔ **Blockchain** : un bonus AGC est crédité automatiquement lors d'une certification réussie. Le cycle lock/release du séquestre est enregistré dans le Ledger.
- **Sécurité** ↔ **Tous** : les transactions atomiques Firestore protègent AGC et Blockchain. L'Audit Trail couvre les quatre axes.

4.5.1 Schéma des flux de données

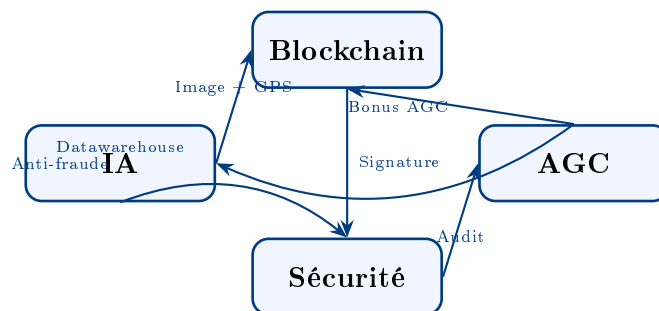


FIGURE 4.7 – Flux de données entre les quatre axes technologiques

4.6 Base de données Firestore

Le modèle de données Firestore est organisé en collections principales et sous-collections pour optimiser les performances et la scalabilité.

Collections Firestore

- **users/{uid}** : informations utilisateur, solde AGC, historique transactions
- **products/{pid}** : produits mis en vente, référence certification
- **certifications/{cid}** : preuves de certification, IPFS CID, txHash
- **transactions/{tid}** : transactions AGC (montant, statut, signature)
- **audit_logs/{lid}** : journal d'audit immuable
- **chat_sessions/{sid}** : sessions de conversation avec Claude

4.6.1 Indexation et requêtes

Stratégie d'indexation

- **Index composite** sur **products** pour les filtres (prix, certification)
- **Index unique** sur **proofHash** pour éviter les doublons
- **Sharding naturel** via Firestore (distribution automatique)
- **Caching** des données fréquentes (solde AGC) via BehaviorSubject Angular

4.7 Scalabilité et déploiement

L'architecture de Jokko Agro est conçue pour évoluer. Plusieurs dimensions de scalabilité sont prises en compte :

Perspectives de scalabilité

- **Firebase Auto-scaling** : Firestore scale automatiquement en fonction du nombre de connexions, sans intervention manuelle.
- **Layer 2 Ethereum** : migration prévue vers **Polygon** pour réduire les frais de transaction (gas fees) lors du passage en production.
- **Multilangue** : l'architecture du chatbot permet d'ajouter d'autres langues (Pulaar, Sérère) sans refonte, en modifiant uniquement le prompt système.
- **Modèle de données extensible** : les collections Firestore sont conçues pour absorber de nouveaux types de certifications sans migration de schéma.
- **CDN et caching** : les images certifiées sont servies via CDN Cloudflare pour une latence minimale en zones rurales.

4.8 Sécurité et authentification

L'architecture de sécurité repose sur une approche multicouche intégrant authentification forte, chiffrement et journalisation immuable.

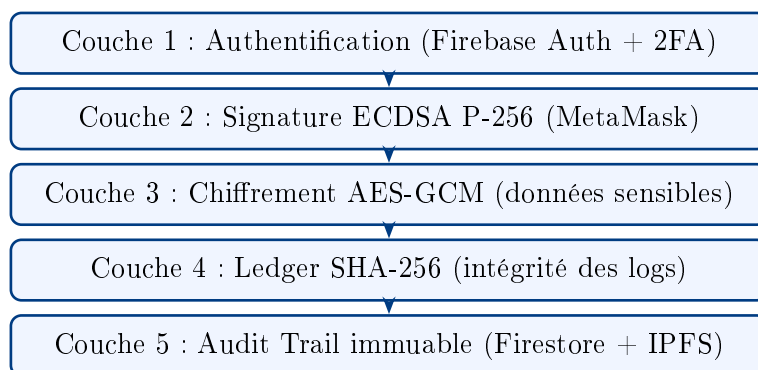


FIGURE 4.8 – Couches de sécurité de Jokko Agro

Mécanismes de sécurité implémentés

- **Authentification forte** : Firebase Auth + authentification à deux facteurs (2FA)
- **Signatures numériques** : ECDSA P-256 pour les transactions AGC et certifications
- **Chiffrement** : AES-GCM pour les données personnelles et métadonnées sensibles
- **Intégrité** : Ledger local SHA-256 pour tracer toutes les modifications
- **Journalisation** : Audit Trail immuable avec preuves IPFS
- **Atomicité** : Transactions Firestore pour les opérations critiques AGC

4.9 Synthèse du chapitre

Ce chapitre a présenté l'architecture technique complète de Jokko Agro, structurée autour de quatre modules interconnectés. Les principaux éléments à retenir sont :

1. Une **architecture modulaire** basée sur Angular 17 avec injection de dépendances, garantissant un couplage faible et une maintenabilité optimale.
2. Des **diagrammes UML** détaillant la structure des classes et les séquences d'interaction critiques (certification blockchain, assistant IA, paiement AGC).
3. Un **modèle de données Firestore** optimisé pour la scalabilité et les requêtes temps réel.
4. Des **mécanismes de scalabilité** anticipant la croissance du nombre d'utilisateurs et le passage en production.
5. Une **architecture de sécurité multicouche** intégrant authentification forte, signatures numériques et journalisation immuable.

Les chapitres suivants approfondiront chaque axe technologique en détaillant leur implémentation, leurs services spécifiques et leurs mécanismes internes.

« Une architecture bien conçue est celle qui permet d'ajouter des fonctionnalités sans en détruire d'autres. »

Axe Intelligence Artificielle

Vue d'ensemble du chapitre

L'axe IA regroupe quatre services : le chatbot agricole bilingue, la reconnaissance vocale en Wolof, la comparaison d'images anti-fraude, et la géolocalisation intelligente. Ensemble, ils rendent Jokko Agro accessible aux agriculteurs quelle que soit leur maîtrise du numérique.

5.1 Présentation de l'axe IA

L'axe IA de Jokko Agro comprend **quatre services Angular** spécialisés, tous orientés vers l'assistance des agriculteurs.

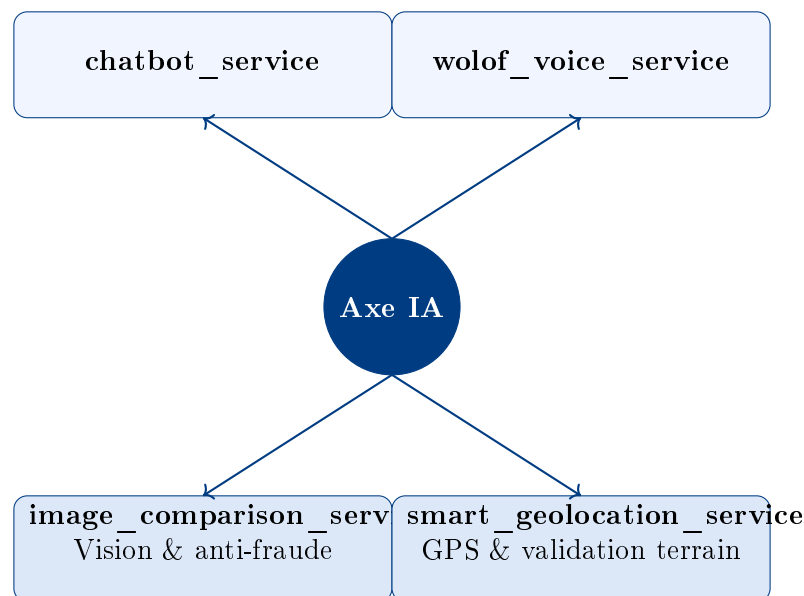


FIGURE 5.1 – Les quatre services de l'axe IA

5.2 Chatbot agricole intelligent

Le chatbot est le point d'entrée principal de l'assistance agricole. Il permet des conversations en **français et en Wolof** pour obtenir des conseils agronomiques personnalisés, adaptés aux cultures et aux conditions climatiques du Sénégal.

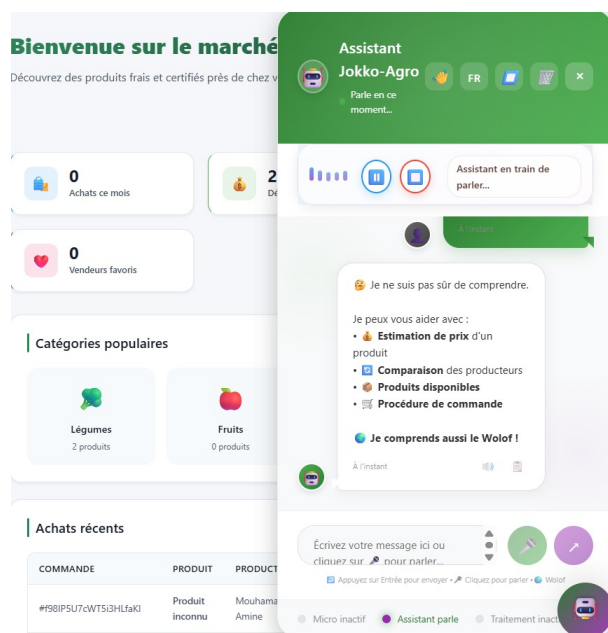


FIGURE 5.2 – Interface du chatbot agricole avec Claude API

Caractéristiques du chatbot

- **Moteur LLM** : API Claude d'Anthropic (claude-3-5-sonnet)
- **Prompt système** : orienté agronomie sénégalaise (cultures locales, saisons, parasites, irrigation)
- **Historique de conversation** : maintenu côté client (array de messages), transmis à chaque requête pour des échanges cohérents sur plusieurs tours
- **Bilingue natif** : le modèle gère nativement le Wolof et répond dans la langue de l'utilisateur

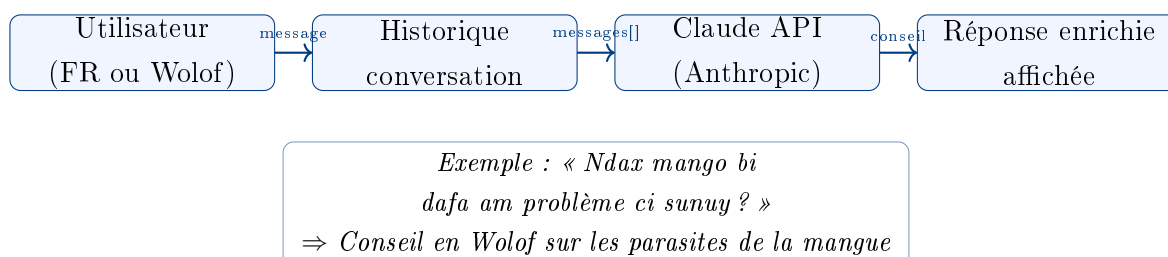


FIGURE 5.3 – Flux de traitement du chatbot agricole

5.3 Reconnaissance vocale (Wolof)

Le service de reconnaissance vocale permet aux agriculteurs analphabètes ou peu à l'aise avec le texte de **dicter leurs questions**.

Le service fonctionne en trois étapes :

1. **Capture audio** : `getUserMedia` + `SpeechRecognition` API pour obtenir une transcription brute.
2. **Post-traitement phonologique** : une couche de normalisation traite les phonèmes spécifiques au Wolof (sons aspirés, tons) absents du support standard de la Web Speech API.
3. **Transmission** : le texte normalisé est envoyé au `chatbot_service` comme s'il avait été saisi manuellement.

Défi technique : La Web Speech API de Google ne supporte pas officiellement le Wolof. Jokko Agro implémente une couche de post-traitement propriétaire qui corrige les erreurs les plus fréquentes dans la transcription automatique du Wolof à partir des modèles français.

5.4 Analyse d'images

Le service `image_comparison_service` est au cœur de l'anti-fraude. Il vérifie la **cohérence visuelle** entre les photos soumises à différentes étapes de certification. Il utilise **TensorFlow.js** (exécuté côté navigateur, sans serveur) et calcule cinq métriques :

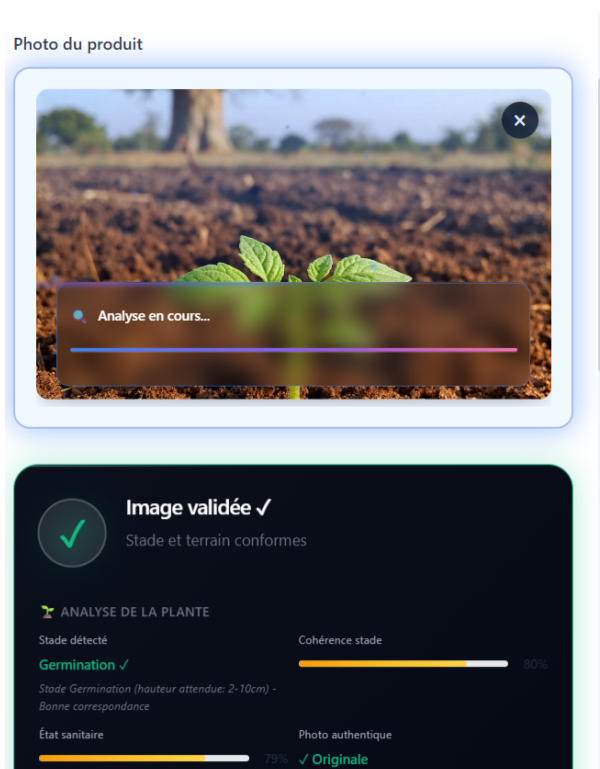


FIGURE 5.4 – Exemple de comparaison d'images anti-fraude avec TensorFlow.js

Métrique	Méthode	Usage	Poids
Similarité struct.	MSE	Détecter les photos copiées	0.30
	Cosinus (embed.)	Identifier le même produit	0.40
Cohérence couleurs	Histogramme HSV	Vérifier maturité / saison	0.20
Cohérence éclairage	Variance pixel	Anti-spoofing	0.10

Le score final est calculé comme :

$$\text{score} = 0.3 \times \text{MSE} + 0.4 \times (1 - \cos) + 0.2 \times \text{HSV} + 0.1 \times \text{var}$$

Selon le score : **accept** (conforme), **review** (vérification manuelle), ou **reject** (fraude probable).

5.5 Géolocalisation intelligente

Le service `smart_geolocation_service` valide que les photos de certification sont bien prises **sur le terrain déclaré**, empêchant les certifications à distance.

Localisation

Latitude: 14.733295

Longitude: -17.269823

0m Excellente

✓ Parfaitement cohérent (16.0m du dernier checkpoint)

FIGURE 5.5 – Validation GPS des points de certification

Mécanismes de validation GPS

- V1. Cohérence de position** : distance entre coordonnées GPS actuelles et coordonnées du checkpoint enregistré $\leq$ rayon de tolérance (500 m).
- V2. Précision GPS** : $\text{accuracy} \leq$ seuil maximal accepté (ex : 100 m) pour rejeter les signaux de mauvaise qualité.
- V3. Anti-spoofing temporel** : cohérence entre l'horodatage GPS (`position.timestamp`) et l'heure réelle du système pour détecter les coordonnées figées ou rejouées.

5.6 Apports de l'IA dans le système

Synthèse de l'axe IA

L'axe IA transforme Jokko Agro en une plateforme réellement accessible : le chatbot supprime la barrière de l'expertise agronomique, la voix élimine la barrière de l'alphabétisation, l'analyse d'images renforce l'intégrité de la certification, et la géolocalisation garantit l'authenticité terrain. Ensemble, ces quatre services rendent la plateforme utilisable par n'importe quel agriculteur sénégalais.

« L'intelligence artificielle ne remplace pas l'agriculteur, elle lui donne les clés pour mieux décider. »

Axe Blockchain et Certification

Vue d'ensemble du chapitre

L'axe Blockchain est le pilier de confiance de Jokko Agro. Il permet d'enregistrer de façon immuable et décentralisée les preuves de certification des produits agricoles, du semis à la récolte, via Ethereum, IPFS et un smart contract Solidity.

6.1 Concept de blockchain

Une **blockchain** est une base de données distribuée dans laquelle les enregistrements (blocs) sont liés cryptographiquement les uns aux autres, formant une chaîne immuable. Toute modification d'un bloc passé invalide tous les blocs suivants, rendant la fraude détectable immédiatement.

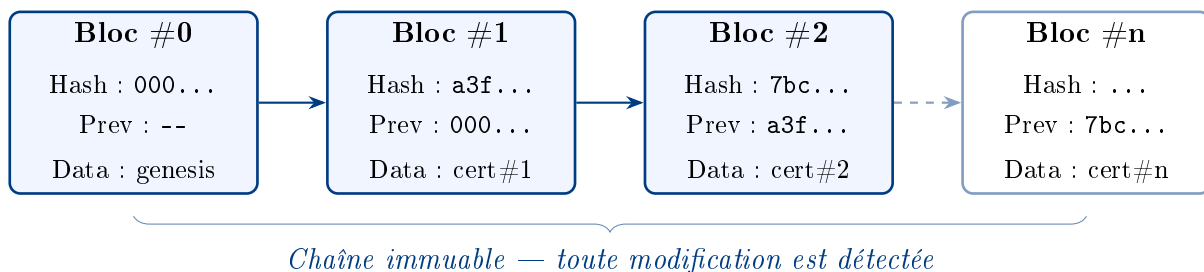


FIGURE 6.1 – Structure d'une blockchain — principe de chaînage par hash

6.2 Smart Contract de certification

Le contrat `CertificationRegistry.sol` est le cœur de la traçabilité. Déployé sur le réseau Ethereum Sepolia, il stocke de façon permanente les preuves de certification avec les garanties suivantes :

Garanties du Smart Contract

- **Immuabilité** : une preuve enregistrée ne peut jamais être modifiée ou supprimée.
- **Non-répudiation** : `msg.sender` (adresse Ethereum de l'appelant) est automatiquement stocké, liant chaque certification à son auteur de façon cryptographiquement prouvable.
- **Horodatage blockchain** : `block.timestamp` est fourni par le réseau, non par l'utilisateur — il ne peut pas être falsifié.
- **Indexation** : l'événement `ProofRegistered` est indexé sur `productId`, permettant des recherches rapides sans parcourir toute la chaîne.

6.3 Enregistrement des preuves

Le flux complet de certification coordonne trois services Angular :

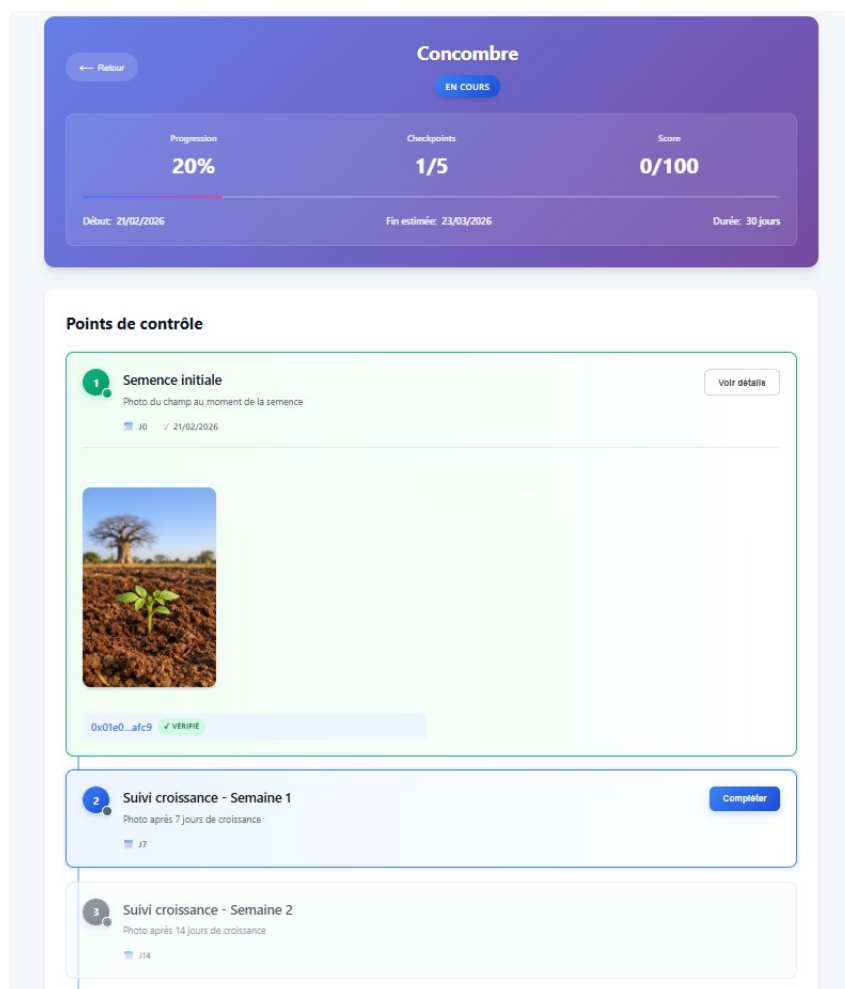


FIGURE 6.2 – Interface de certification avec upload d'image

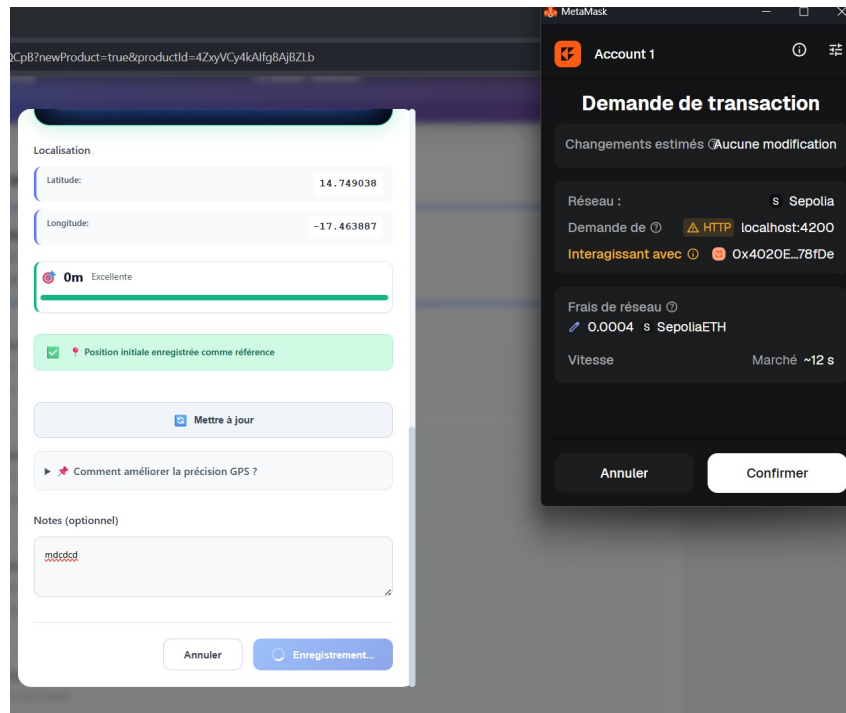


FIGURE 6.3 – Popup MetaMask pour signature de la transaction

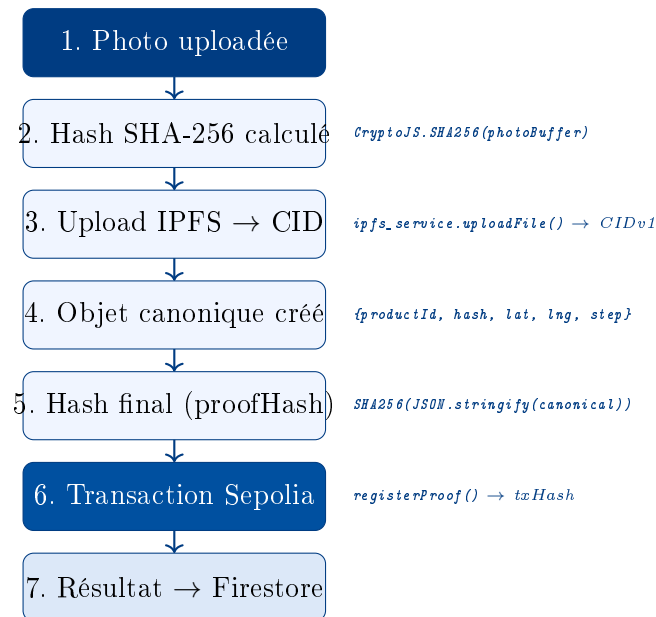


FIGURE 6.4 – Flux complet de création d'une preuve de certification

6.4 Interaction avec Ethereum

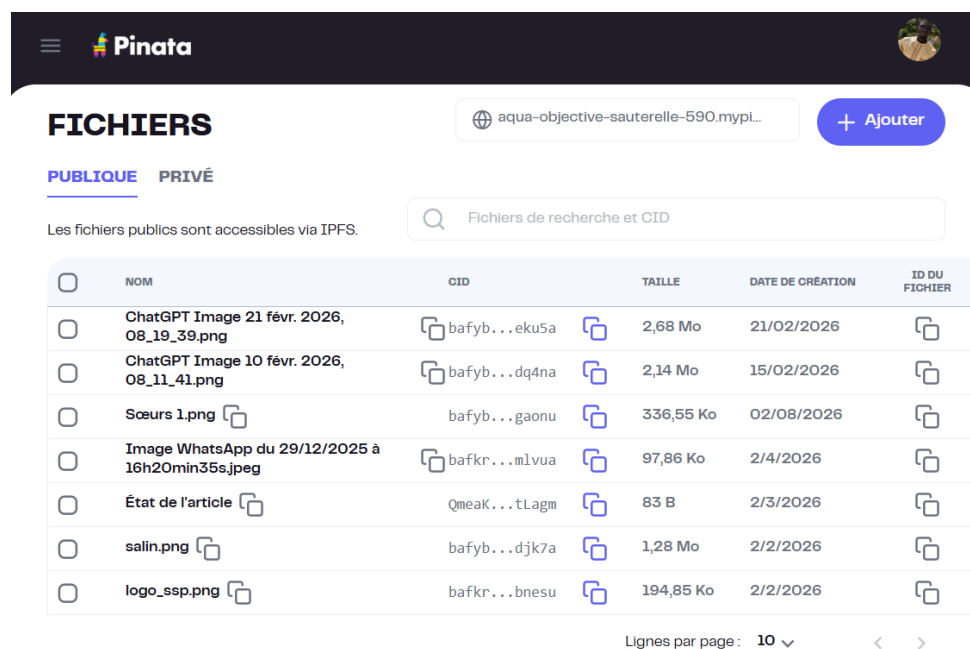
Le service `ethereum_service` gère toutes les interactions avec MetaMask et le réseau Ethereum via Alchemy RPC :

Méthodes clés de l'ethereum_service

- `connectWallet()` — `eth_requestAccounts` → popup MetaMask
- `checkNetwork(11155111)` — vérifie que l'utilisateur est sur Sepolia
- `switchNetwork(11155111)` — bascule automatiquement sur Sepolia
- `encodeContractCall(ABI, 'registerProof', params)` — encodage ABI
- `sendContractTransaction(addr, data)` — envoi transaction signée
- `getTransactionReceipt(txHash)` — suivi confirmation (status 0x1)

6.5 Stockage IPFS

Les photos de certification sont stockées sur **IPFS via Pinata**, garantissant leur accessibilité décentralisée. Le CIDv1 produit des identifiants stables, résistants aux collisions.



The screenshot shows the Pinata web interface. At the top, there's a header with the Pinata logo and a user profile picture. Below the header, there's a section titled "FICHIERS" with a search bar and a "+ Ajouter" button. The main content area displays a table of uploaded files. The table has columns for "NOM", "CID", "TAILLE", "DATE DE CRÉATION", and "ID DU FICHIER". The files listed include "ChatGPT Image 21 févr. 2026, 08_19_39.png", "ChatGPT Image 10 févr. 2026, 08_11_41.png", "Sœurs 1.png", "Image WhatsApp du 29/12/2025 à 16h20min35s.jpeg", "État de l'article", "salin.png", and "logo_ssp.png". Each row includes a checkbox, the file name, the CID, the size, the creation date, and a link to the file.

	NOM	CID	TAILLE	DATE DE CRÉATION	ID DU FICHIER
☐	ChatGPT Image 21 févr. 2026, 08_19_39.png	bafyb...eku5a	2,68 Mo	21/02/2026	
☐	ChatGPT Image 10 févr. 2026, 08_11_41.png	bafyb...dq4na	2,14 Mo	15/02/2026	
☐	Sœurs 1.png	bafyb...gaonu	336,55 Ko	02/08/2026	
☐	Image WhatsApp du 29/12/2025 à 16h20min35s.jpeg	bafkr...mlvua	97,86 Ko	2/4/2026	
☐	État de l'article	QmeaK...tLagm	83 B	2/3/2026	
☐	salin.png	bafyb...djk7a	1,28 Mo	2/2/2026	
☐	logo_ssp.png	bafkr...bnesu	194,85 Ko	2/2/2026	

Lignes par page: 10

FIGURE 6.5 – Confirmation d'upload IPFS avec CID généré

Méthodes de l'ipfs_service

- `uploadFile(file)` → {cid, url} — photo vers IPFS
- `uploadJSON(data)` → {cid, url} — métadonnées vers IPFS
- `verifyCID(cid)` → boolean — vérification existence
- `getGatewayUrl(cid)` → string — URL CDN Pinata

6.6 Traçabilité et transparence

La certification s'organise en **7 checkpoints** couvrant le cycle complet de production agricole, avec un système de badges récompensant la qualité :

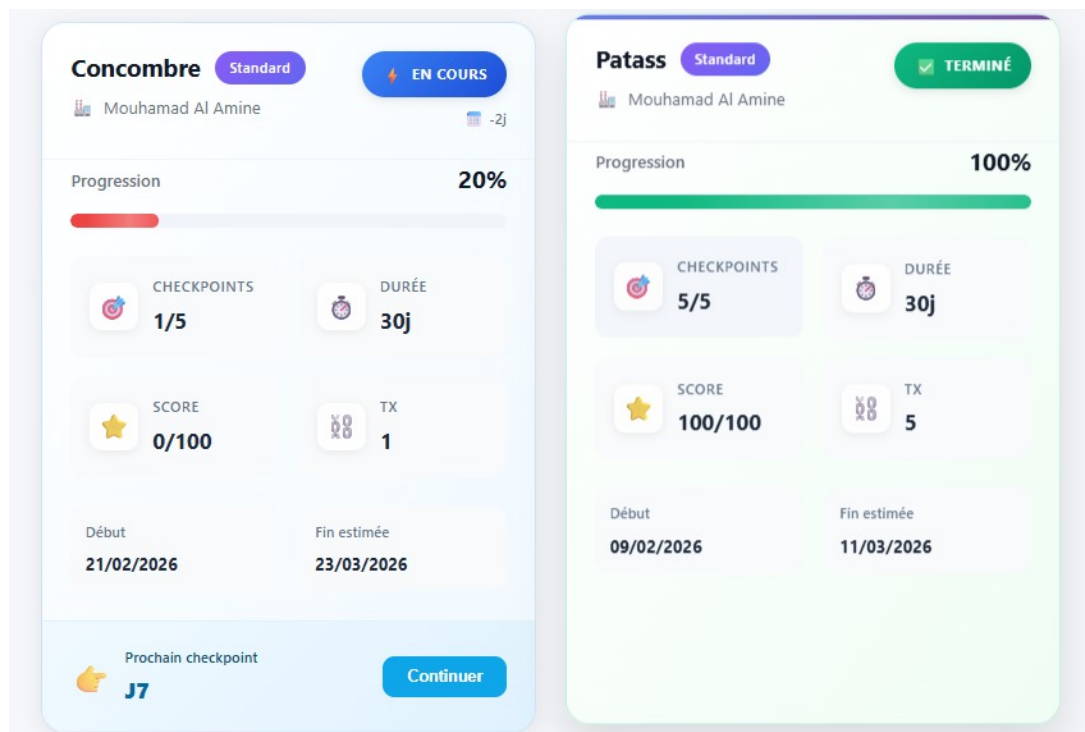


FIGURE 6.6 – Système de badges (Bronze/Silver/Gold) selon le niveau de certification

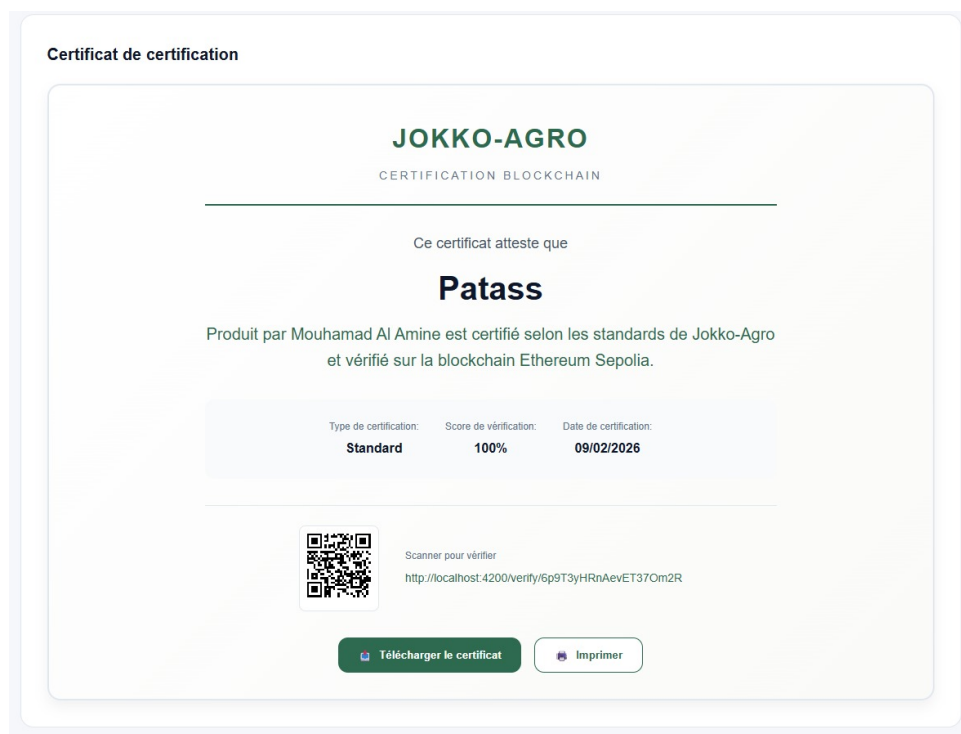


FIGURE 6.7 – Détail d'un certificat blockchain avec QR code de vérification

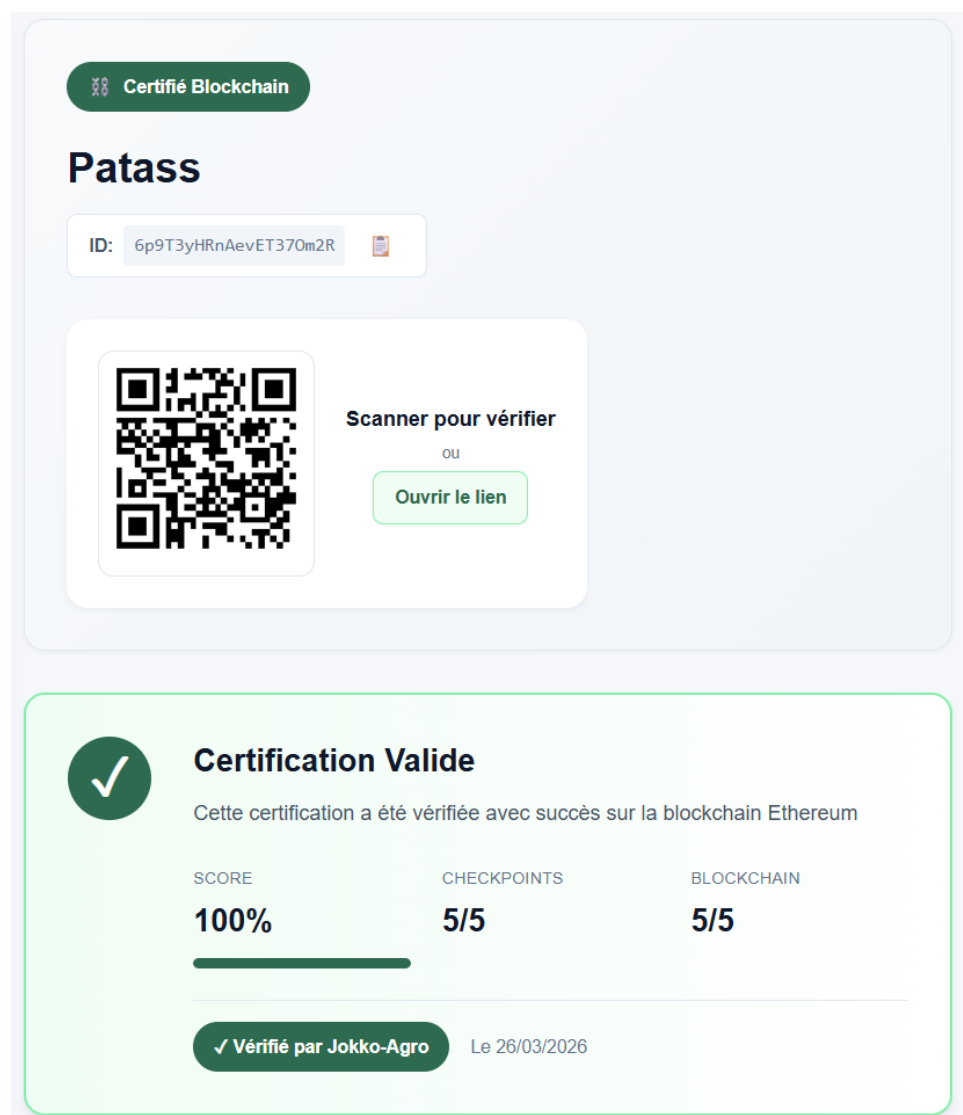


FIGURE 6.8 – QR code permettant la vérification instantanée de la certification

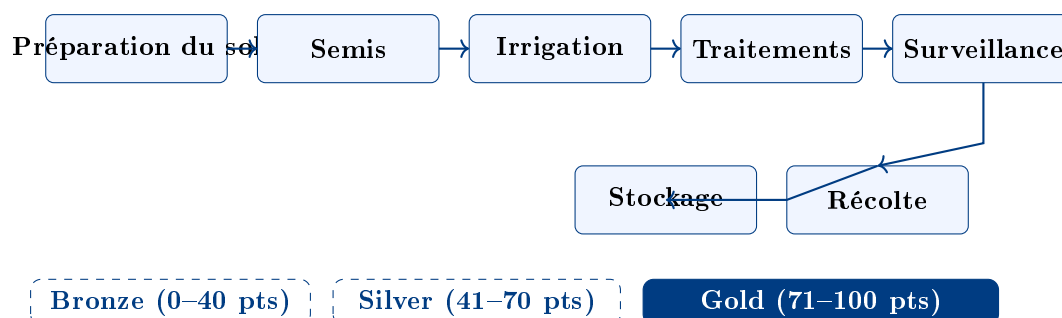


FIGURE 6.9 – Les 7 checkpoints de certification et le système de badges

« La blockchain ne crée pas la confiance, elle la rend vérifiable par tous. »

Axe Agricoins (AGC) et Datawarehouse

Vue d'ensemble du chapitre

L'axe Agricoins crée un écosystème économique incitatif au sein de Jokko Agro. Il combine une monnaie numérique agricole (AGC), un système de paiement hybride sécurisé, un module commerce complet et un datawarehouse d'aide à la décision.

7.1 Présentation des Agricoins

L'**Agricoins (AGC)** est la monnaie numérique interne de Jokko Agro. Sa création répond à trois objectifs : simplifier les micro-transactions agricoles, inciter les bonnes pratiques via des récompenses, et créer un lien direct entre certification et valeur économique.

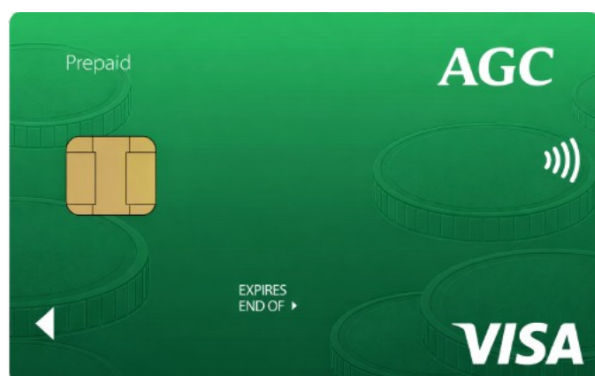


FIGURE 7.1 – Portefeuille AGC

Parité et sources d'AGC

1 AGC = 100 FCFA (parité fixe)

- **Achat direct** : via Wave, Orange Money ou Free
- **Récompense de certification** : bonus AGC crédité automatiquement à chaque checkpoint blockchain validé
- **Solde initial** : émission à l'inscription pour les nouveaux utilisateurs

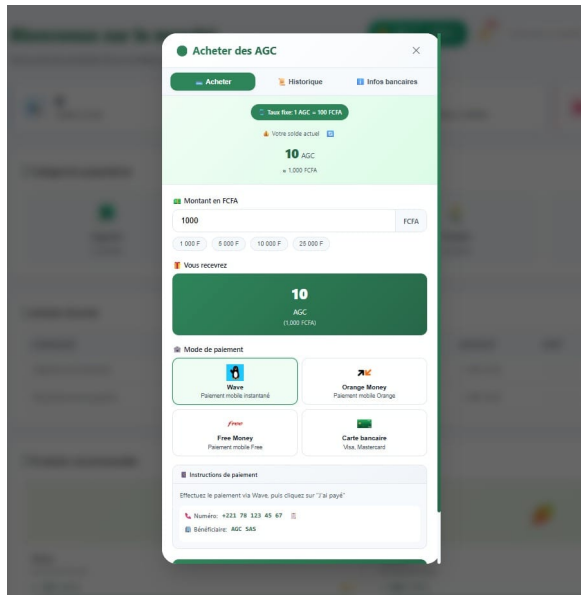


FIGURE 7.2 – Interface d'achat des Agri-coins (AGC)

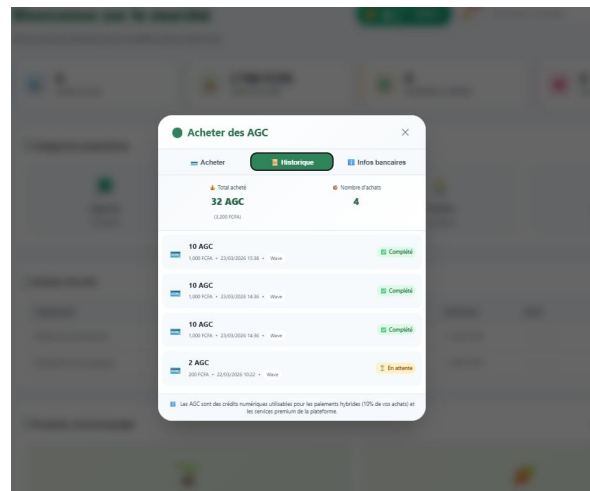


FIGURE 7.3 – Historique des transactions AGC

L'interface d'achat permet aux utilisateurs d'acquérir des AGC via les plateformes de mobile money (Wave, Orange Money, Free) ou par carte bancaire. Le taux de conversion est fixe : **1 AGC = 100 FCFA**. L'historique des transactions permet de suivre l'ensemble des achats effectués, avec le statut de chaque transaction (Complété / En attente).

7.2 Mécanisme d'émission

Le solde AGC de chaque utilisateur est stocké dans Firestore sous la structure suivante :

Structure Firestore du portefeuille AGC

```

1 users/{uid}/
2   agc_balance: number           // Solde courant en AGC
3   agc_transactions[]:
4     type: 'credit' | 'debit'
5     amount: number             // Montant en AGC
6     fiatAmount: number         // Equivalent FCFA
7     timestamp: Timestamp
8     reason: string             // 'CERTIFICATION_REWARD' | '
9     txHash?: string            // Hash ledger si applicable

```

7.3 Paiement hybride (AGC + Mobile Money)

Le système de paiement hybride permet de combiner AGC et mobile money pour régler une commande. Un mécanisme de **séquestre (escrow)** protège acheteur et vendeur.

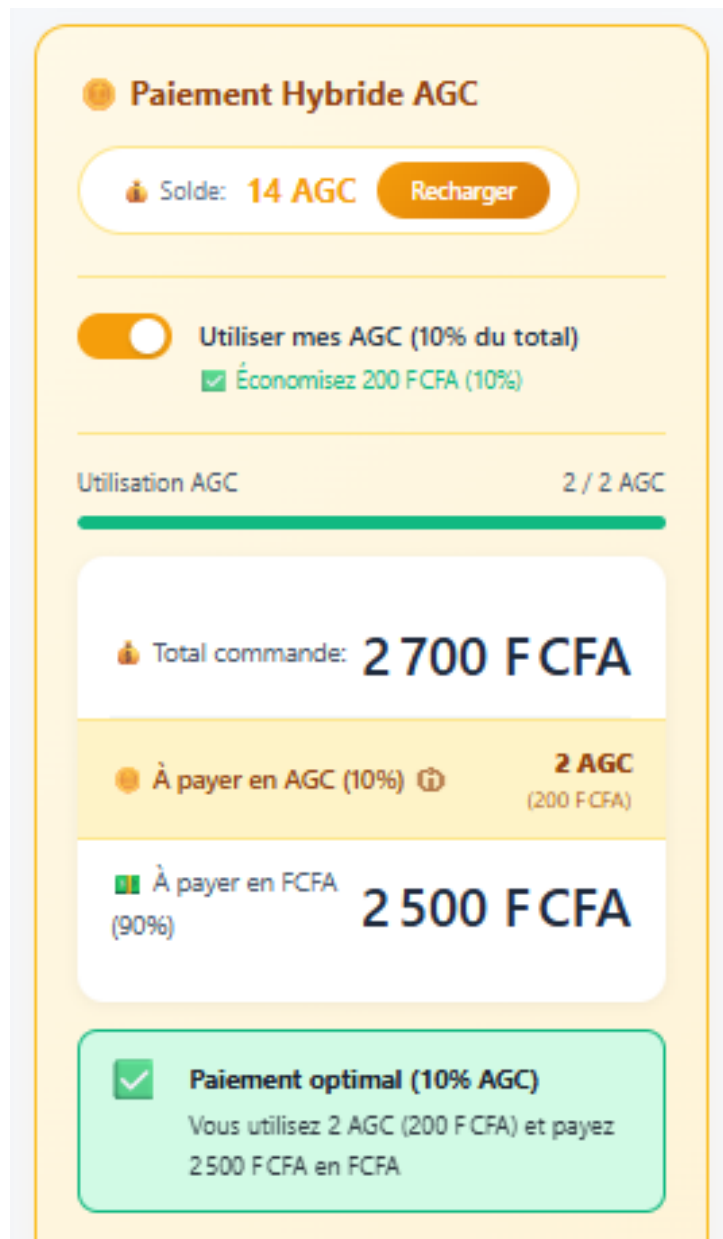


FIGURE 7.4 – Interface de paiement avec Wave / Orange Money

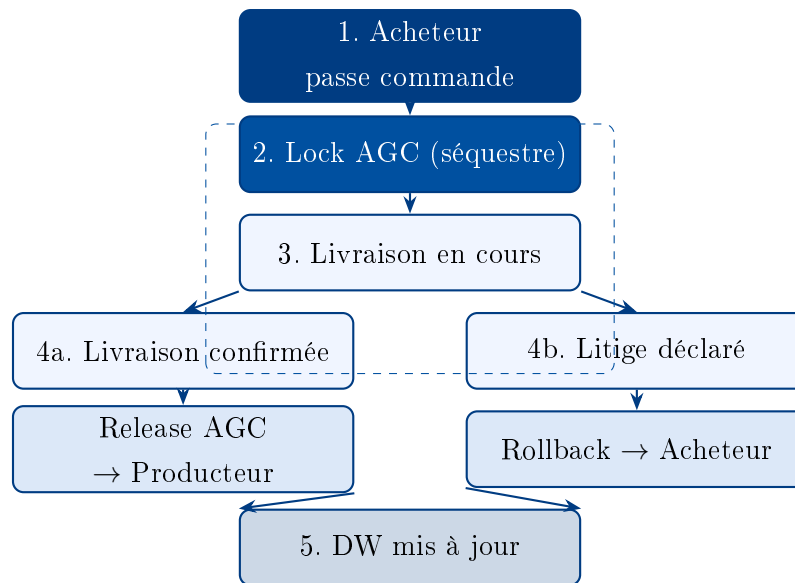


FIGURE 7.5 – Cycle de vie d'un paiement AGC avec escrow

7.4 Gestion des transactions

Toutes les transactions AGC utilisent les **transactions atomiques Firestore** (`runTransaction`), garantissant que le débit de l'acheteur et le crédit du producteur s'effectuent toujours ensemble — ou pas du tout en cas d'erreur réseau.

Propriétés ACID garanties : **Atomicité** (débit + crédit simultanés), **Cohérence** (double dépense impossible, balance vérifiée atomiquement), **Rollback automatique** (état précédent restauré en cas d'erreur).

7.5 Commerce et marketplace

Le module commerce gère le cycle complet d'une vente avec **7 états de commande** :

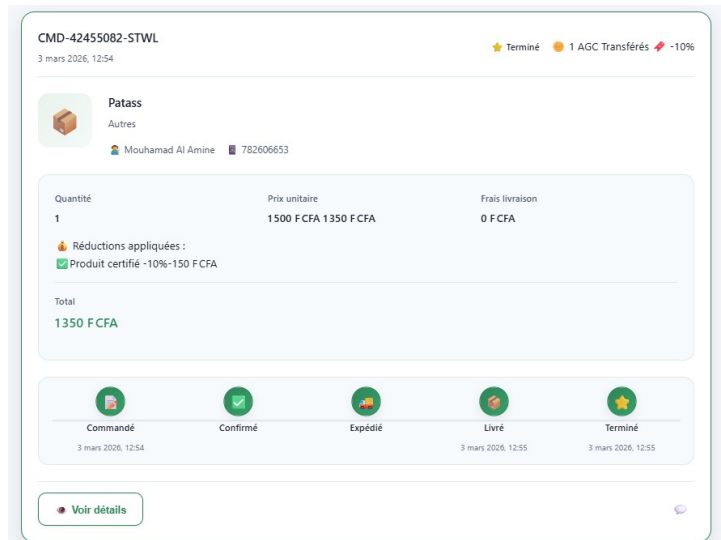


FIGURE 7.6 – Interface du marketplace avec produits certifiés

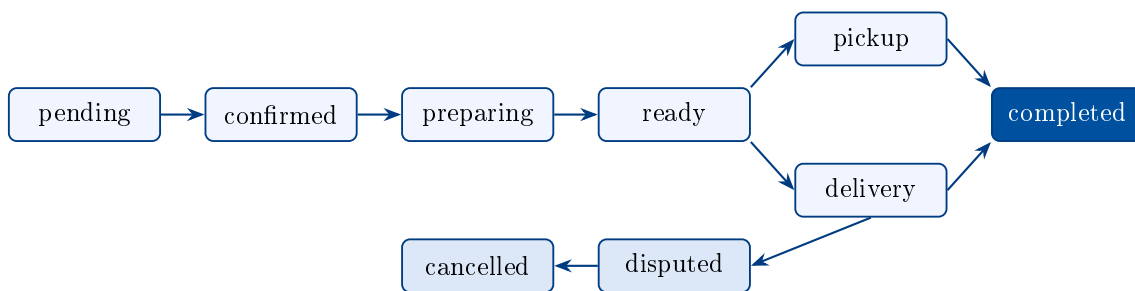


FIGURE 7.7 – Machine à états d'une commande Jokko Agro

Axe Sécurité et Cryptographie

Vue d'ensemble du chapitre

L'axe Sécurité est transversal à toute la plateforme. Il garantit l'intégrité des données, l'authentification des acteurs et la confidentialité des informations sensibles à travers dix mécanismes cryptographiques complémentaires, organisés en couches de défense en profondeur (Defence in Depth). Chaque couche adresse une menace spécifique et les couches sont conçues pour se renforcer mutuellement.

8.1 Architecture de sécurité

La sécurité de Jokko Agro repose sur une architecture en **dix couches complémentaires**, chacune adressant une menace spécifique. Cette approche de défense en profondeur garantit que la compromission d'une couche n'entraîne pas celle de l'ensemble du système.

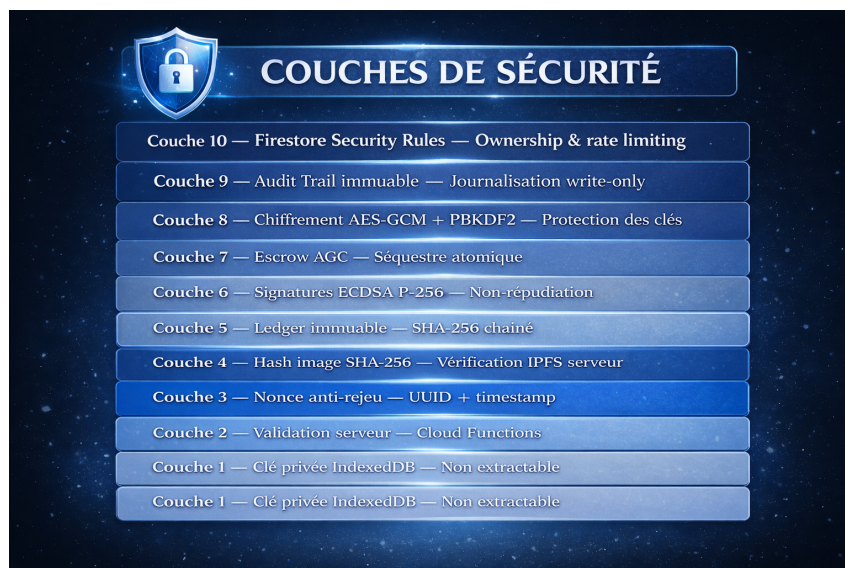


FIGURE 8.1 – Architecture de sécurité en dix couches de Jokko Agro

8.2 Tableau récapitulatif des dix couches

Les dix couches de sécurité de Jokko Agro

#	Couche	Menace adressée	Technologies
C1	Signatures ECDSA P-256	Répudiation de transaction	ECDSA P-256 · WebCrypto API · JSON canonique
C2	IndexedDB non extractable	Vol de clé par injection XSS	WebCrypto API · IndexedDB Extractable : false
C3	Validation serveur (Cloud Functions)	Contournement des règles client	Firebase Cloud Functions · runTransaction
C4	Nonce anti-rejeu (UUID + timestamp)	Replay attack	crypto.randomUUID() · Firestore used_nonces · TTL
C5	Hash image SHA-256 (vérification IPFS)	Manipulation du score IA	WebCrypto SHA-256 · IPFS Pinata Gateway
C6	Ledger immuable (SHA-256 chaîné)	Falsification rétroactive	SHA-256 · Firestore chaînage previousHash
C7	Chiffrement AES-GCM + PBKDF2	Vol de clé privée par accès Firestore	AES-GCM-256 · PBKDF2-SHA256 310k iter.
C8	Escrow AGC (séquestre atomique)	Double-dépense	Firestore runTransaction · Lock/Release/Cancel
C9	Audit Trail immuable	Effacement de preuves post-compromission	Firestore write-only Rules · serverTimestamp
C10	Firestore Security Rules	Accès non autorisé aux données	Ownership Rules · Admin SDK only

8.3 Description détaillée des couches

8.3.1 C1 — Signatures ECDSA P-256 (Web Crypto API)

Couche C1 — Signatures ECDSA P-256

- **Composant** : `CryptoECDSAService` (`crypto-ecdsa.service.ts`)
- **Menace adressée** : Répudiation de transaction — « je n'ai pas signé ça »
- **Technologies** : ECDSA P-256 · WebCrypto API · Signature JSON canonique

f Toutes les transactions financières (lock AGC, release, transfer) sont signées avec la courbe elliptique P-256 via l'API WebCrypto native du navigateur, sans dépendance externe. La signature couvre le payload JSON canonique de la transaction — toute modification du contenu invalide la signature. La clé publique est stockée dans Firestore (collection `user_keys`) et utilisée pour la vérification. La vérification est également effectuée côté serveur via Cloud Function (C3).

8.3.2 C2 — IndexedDB non extractable

Couche C2 — IndexedDB non extractable

- **Composant** : `SecureKeyService` (`secure-key.service.ts`)
- **Menace adressée** : Vol de clé par injection XSS (Cross-Site Scripting)
- **Technologies** : WebCrypto API · IndexedDB Extractable : `false` · P-256

La clé privée ECDSA est générée avec l'option `extractable: false` via l'API WebCrypto du navigateur et stockée directement dans IndexedDB sous forme d'objet `CryptoKey`. Contrairement à `sessionStorage` (accessible à tout script JS), un objet `CryptoKey` non extractable ne peut jamais être sérialisé ni exporté en bytes — même du code malveillant injecté via XSS peut appeler `sign()` mais ne verra jamais les octets de la clé privée. C'est l'équivalent logiciel d'un Secure Enclave ou HSM (Hardware Security Module).

8.3.3 C3 — Validation serveur (Cloud Functions)

Couche C3 — Validation serveur Cloud Functions

- **Composant** : `certify-checkpoint.ts` (`functions/src/`)
- **Menace adressée** : Contournement des règles client Angular via API directe
- **Technologies** : Firebase Cloud Functions Node.js · `runTransaction`

Toute la logique métier critique est déplacée dans une Cloud Function Firebase (Node.js), hors de portée du navigateur. La fonction `certifyCheckpoint()` re-vérifie indépendamment : l'authentification Firebase, la signature ECDSA via la clé publique Firestore, l'appartenance du produit à l'utilisateur, le rate limiting (1 checkpoint/60s) et la validité des données. Un utilisateur ne peut pas bypasser Angular en envoyant directement des requêtes à l'API Firebase — sans passer par la Cloud Function, aucune certification n'est enregistrée. Toutes les écritures sont atomiques (`runTransaction`) avec audit trail immuable.

8.3.4 C4 — Nonce anti-rejeu (UUID + timestamp)

Couche C4 — Nonce anti-rejeu

- **Composant** : `SignedMessageService` (`signed-message.service.ts`)
- **Menace adressée** : Replay attack — rejeu d'une signature interceptée
- **Technologies** : `crypto.randomUUID()` · Firestore `used_nonces` · TTL 10 min

Chaque message signé embarque un UUID v4 aléatoire (nonce) généré via `crypto.randomUUID()` ainsi qu'un timestamp Unix en secondes. Côté Cloud Function, le nonce est vérifié dans la collection Firestore `used_nonces` : s'il existe déjà, la requête est immédiatement rejetée (`code already-exists`). Si c'est la première utilisation, le nonce est marqué comme consommé avec un TTL de 10 minutes. Le timestamp limite la fenêtre d'attaque à ± 5 minutes — une signature capturée perd toute validité après ce délai. Cette combinaison nonce + timestamp rend toute attaque par rejeu impossible même sur un réseau intercepté.

8.3.5 C5 — Hash image SHA-256 (vérification IPFS serveur)

Couche C5 — Hash image SHA-256

- **Composant** : ImageIntegrityService (image-integrity.service.ts)
- **Menace adressée** : Manipulation du score TensorFlow.js avant envoi au serveur
- **Technologies** : WebCrypto SHA-256 · IPFS Pinata Gateway · Node crypto

Le scoring de qualité TensorFlow.js s'exécute dans le navigateur et peut être manipulé par un utilisateur avancé. La parade consiste à calculer le hash SHA-256 de l'image brute *avant* toute analyse IA, le signer et l'envoyer avec la preuve de certification. Côté Cloud Function, l'image est re-téléchargée depuis son CID IPFS via le gateway Pinata, son hash SHA-256 est recalculé de manière indépendante et comparé au hash soumis. Si les deux hash diffèrent, l'image a été modifiée après le calcul initial — la certification est immédiatement rejetée. Le score IA reste un indicateur (marqué `indicative_only` dans le payload) mais l'intégrité de l'image est prouvée cryptographiquement.

8.3.6 C6 — Ledger immuable (SHA-256 chaîné)

Couche C6 — Ledger immuable SHA-256

- **Composant** : BlockchainService (ledger-service.ts)
- **Menace adressée** : Falsification rétroactive des transactions AGC
- **Technologies** : SHA-256 · Firestore chaînage previousHash

Un ledger local inspiré de la blockchain est implémenté dans Firestore pour toutes les transactions AGC. Chaque bloc contient le hash SHA-256 du bloc précédent (previousHash), sa hauteur (blockHeight), un nonce anti-collision, un timestamp et les données de transaction. Son hash est calculé via :

$$\text{hash} = \text{SHA256}(\text{previousHash} + \text{data} + \text{timestamp} + \text{nonce})$$

Cette structure rend toute modification rétroactive détectable : changer un bloc invalide tous les blocs suivants. Un service de vérification périodique parcourt l'ensemble du ledger et alerte en cas d'incohérence de hash, avec journalisation critique dans l'Audit Trail.

8.3.7 C7 — Chiffrement AES-GCM + PBKDF2 (310 000 itérations)

Couche C7 — Chiffrement AES-GCM + PBKDF2

- **Composant** : UserKeysService (user-keys.service.ts)
- **Menace adressée** : Vol de clé privée par accès à Firestore
- **Technologies** : AES-GCM-256 · PBKDF2-SHA256 310k iter. · OWASP 2024

La clé privée ECDSA est chiffrée avec AES-GCM 256 bits avant tout stockage dans Firestore. La clé de chiffrement est dérivée du mot de passe utilisateur via PBKDF2 avec 310 000 itérations SHA-256 et un salt aléatoire de 16 octets — recommandation OWASP 2024. AES-GCM offre un chiffrement authentifié : toute modification du chiffré (même 1 bit) fait échouer le déchiffrement (authentication tag invalide). Le format stocké est :

$$\text{base64}(\text{salt}[16] \parallel \text{IV}[12] \parallel \text{ciphertext})$$

Cette couche protège la clé privée chiffrée en transit et en stockage.

8.3.8 C8 — Escrow AGC (séquestre atomique)

Couche C8 — Escrow AGC

- **Composant** : AgcService · SalesService (escrow-lock.ts)
- **Menace adressée** : Double-dépense · fonds non reversés en cas de litige
- **Technologies** : Firestore runTransaction · Escrow Lock/Release/Cancel

Les AGC (Agricoins) impliqués dans une transaction commerciale sont bloqués dans un séquestre Firestore dès la confirmation de commande. Quatre états sont possibles :

- **Lock** : signature ECDSA requise, AGC gelés atomiquement
- **Release** : hash Ledger enregistré, AGC transférés vers le producteur après confirmation livraison
- **Cancel** : rollback Firestore atomique, AGC remboursés à l'acheteur
- **Litige** : intervention manuelle avec traces Audit Trail complètes

Toutes les opérations utilisent `runTransaction()` Firestore pour garantir l'atomicité — une coupure réseau en cours de transaction ne laisse jamais les fonds dans un état incohérent. La couche C4 (nonce) protège les opérations de lock contre les rejeux. Délai maximum : 7 jours pour confirmation, 48h pour résolution de litige.

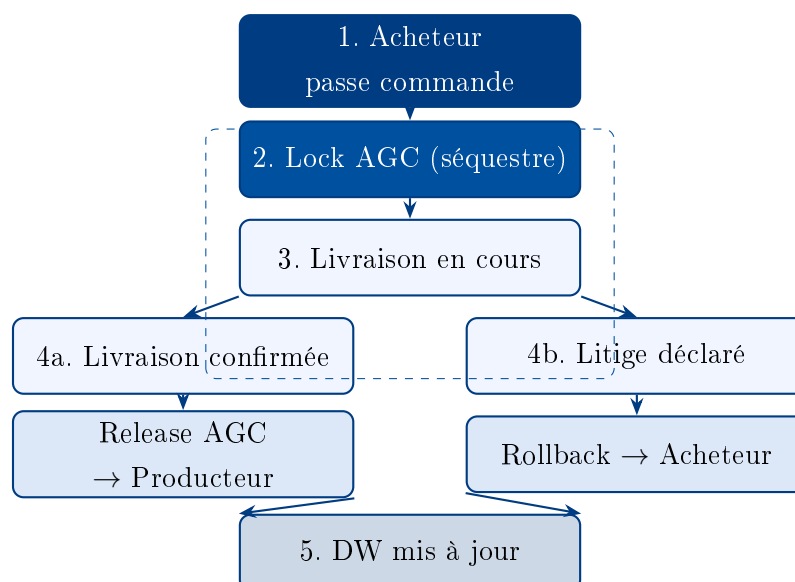


FIGURE 8.2 – Cycle de vie d'un paiement AGC avec escrow

8.3.9 C9 — Audit Trail immuable (journalisation write-only)

Couche C9 — Audit Trail immuable

- **Composant** : `AuditTrailService` (`audit-trail.service.ts`)
- **Menace adressée** : Effacement de preuves post-compromission
- **Technologies** : Firestore write-only Rules · `serverTimestamp` · Admin SDK

Chaque action sensible (génération de clés, lock/release AGC, certification checkpoint, signature invalide, tentative de double-dépense, falsification détectée) est enregistrée dans la collection Firestore `audit_logs` avec `serverTimestamp()` côté serveur. Les Security Rules Firestore interdisent formellement toute modification ou suppression :

```
1 match /audit_logs/{logId} {  
2   allow create: if request.auth != null;  
3   allow read:   if request.auth.token.admin == true;  
4   allow update, delete: if false; // JAMAIS modifiable  
5 }
```

Même un administrateur ne peut pas supprimer un log. Les entrées critiques incluent le `blockHeight` et `blockHash` du ledger pour une traçabilité croisée.

8.3.10 C10 — Firestore Security Rules (ownership + rate limiting)

Couche C10 — Firestore Security Rules

- **Composant** : `firestore.rules` · Cloud Functions middleware
- **Menace adressée** : Accès non autorisé aux données d'autres utilisateurs
- **Technologies** : Firestore Security Rules · Ownership · Admin SDK only

Les règles Firestore constituent le dernier rempart de sécurité côté serveur. Chaque collection est protégée par des règles d'ownership : un utilisateur ne peut lire ou modifier que ses propres documents (`request.auth.uid == resource.data.userId`). Les produits et certifications vérifient l'appartenance au producteur. Les AGC et transactions financières n'autorisent l'écriture que via les Cloud Functions (Admin SDK), jamais directement depuis le client. La couche C3 complète ces règles avec un rate limiting applicatif (1 checkpoint/60s) et une validation métier approfondie impossible à implémenter dans les Security Rules seules. Les règles sont versionnées et auditées à chaque déploiement.

```
// =====
// COLLECTION : users
// Structure : { uid, fullName, phone, email, role, location, createdAt }
// =====
match /users/{uid} {
  // Lecture : chacun peut lire son propre profil
  allow read: if isOwner(uid) || isAuth();

  // Création : uniquement à l'inscription, par soi-même
  allow create: if isOwner(uid)
  && isNonEmptyString(request.resource.data.fullName)
  && isNonEmptyString(request.resource.data.email)
  && request.resource.data.role in ['producer', 'buyer']
  && request.resource.data.uid == uid;

  // Mise à jour : chacun peut modifier son propre profil
  allow update: if isOwner(uid)
  && request.resource.data.role == resource.data.role
  && request.resource.data.uid == resource.data.uid;

  // Suppression : interdite
  allow delete: if false;

  // --- Sous-collection : notifications
  match /notifications/{notifId} {
    allow read: if isOwner(uid);
    allow create: if isAuth();
    allow update: if isOwner(uid);
    allow delete: if isOwner(uid);
  }
}

// =====
// COLLECTION : products
// =====
match /products/{productId} {
  allow read: if isAuth();

  allow create: if isProducer()
  && request.resource.data.productId == request.auth.uid
  && isNonEmptyString(request.resource.data.name)
  && isPositiveNumber(request.resource.data.price)
  && request.resource.data.stock is number;

  allow update: if isProductOwner()
  && request.resource.data.productId == resource.data.productId;

  allow delete: if isProductOwner();
}
```

FIGURE 8.3 – Extrait des règles de sécurité Firestore

```
// =====
// COLLECTION : users
// Structure : { uid, fullName, phone, email, role, location, createdAt }
// =====
match /users/{uid} {
  // Lecture : chacun peut lire son propre profil
  allow read: if isOwner(uid) || isAuth();

  // Création : uniquement à l'inscription, par soi-même
  allow create: if isOwner(uid)
  && isNonEmptyString(request.resource.data.fullName)
  && isNonEmptyString(request.resource.data.email)
  && request.resource.data.role in ['producer', 'buyer']
  && request.resource.data.uid == uid;

  // Mise à jour : chacun peut modifier son propre profil
  allow update: if isOwner(uid)
  && request.resource.data.role == resource.data.role
  && request.resource.data.uid == resource.data.uid;

  // Suppression : interdite
  allow delete: if false;

  // --- Sous-collection : notifications
  match /notifications/{notifId} {
    allow read: if isOwner(uid);
    allow create: if isAuth();
    allow update: if isOwner(uid);
    allow delete: if isOwner(uid);
  }
}

// =====
// COLLECTION : products
// =====
match /products/{productId} {
  allow read: if isAuth();

  allow create: if isProducer()
  && request.resource.data.productId == request.auth.uid
  && isNonEmptyString(request.resource.data.name)
  && isPositiveNumber(request.resource.data.price)
  && request.resource.data.stock is number;

  allow update: if isProductOwner()
  && request.resource.data.productId == resource.data.productId;

  allow delete: if isProductOwner();
}
```

FIGURE 8.4 – Extrait des règles de sécurité Firestore

8.4 Scénarios d'attaque et complémentarité des couches

Les dix couches forment un système défensif en profondeur (Defence in Depth). Aucune couche n'est suffisante seule, mais leur combinaison garantit qu'un attaquant qui contourne une couche se heurte immédiatement à la suivante.

Scénarios d'attaque et couches activées

Scénario d'attaque	Couches qui s'activent
XSS — Injection JavaScript	C2 (IndexedDB non extractable) + C1 (signature requise) + C3 (vérif. serveur)
Replay attack	C4 (nonce UUID + timestamp) + C3 (vérif. Firestore used_nonces)
Manipulation image IA	C5 (hash SHA-256 IPFS) + C3 (recalcul serveur indépendant)
Double-dépense AGC	C8 (escrow atomique) + C6 (ledger chaîné) + C9 (audit trail)
Accès API directe (bypass Angular)	C3 (Cloud Functions) + C10 (Firestore Rules) + C1 (ECDSA obligatoire)
Falsification de transaction	C6 (ledger SHA-256 chaîné) + C1 (signature) + C9 (audit) + C3 (vérif. serveur)
Effacement de preuves	C9 (write-only Firestore Rules : allow delete : if false)
Vol de clé Firestore	C7 (AES-GCM chiffrement) + C2 (clé privée jamais stockée en clair)

8.5 Tests de sécurité automatisés

Pour valider l'implémentation des dix couches de sécurité, un **tableau de bord de test** a été développé. Il exécute automatiquement 25 tests couvrant l'ensemble des mécanismes cryptographiques et métier de la plateforme.

8.5.1 Architecture du tableau de bord

Le tableau de bord est un composant Angular autonome (`SecurityDashboardComponent`) qui orchestre l'exécution séquentielle des tests. Chaque test est isolé et produit des résultats détaillés avec métriques de performance.

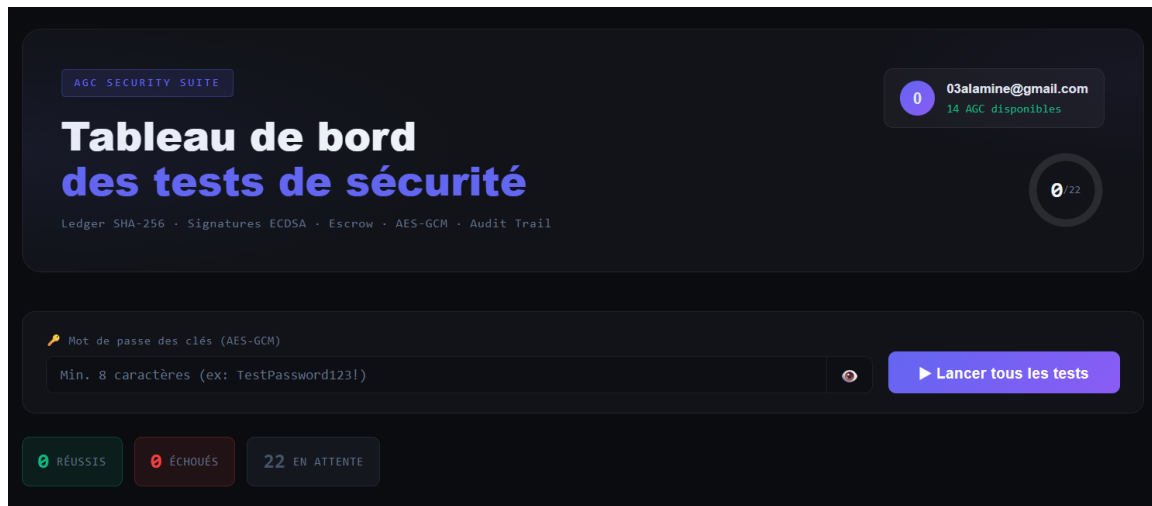


FIGURE 8.5 – Tableau de bord des tests de sécurité — Vue d'ensemble

8.5.2 Jeu de tests implémentés

Les 25 tests sont organisés en 5 catégories correspondant aux couches de sécurité :

Tests des signatures ECDSA (Couche C1)

- **Génération paire P-256** : Création clé privée/publique avec WebCrypto
- **Signature transaction** : Génération de signature ECDSA
- **Vérification valide** : Validation d'une signature authentique
- **Rejet falsification** : Détection de modification de transaction
- **Audit complet** : Vérification de toutes les signatures du ledger

Tests du stockage IndexedDB (Couche C2)

- **Génération clé non extractable** : Vérification extractable : false
- **Stockage IndexedDB** : Persistance des clés entre sessions
- **Restauration session** : Récupération clé après fermeture navigateur

Tests du Ledger immuable (Couche C6)

- **Hash SHA-256** : Vérification du déterminisme et de l'effet avalanche
- **Chaînage previousHash** : Validation de la liaison cryptographique
- **Intégrité complète** : Parcours et validation de l'ensemble du ledger
- **Détection falsification** : Modification d'un bloc et vérification d'invalidation

Tests du séquestre AGC (Couche C8)

- **Lecture solde** : Consultation des AGC disponibles et bloqués
- **Blocage (lock)** : Gel atomique des fonds en Firestore
- **Vérification statut** : Confirmation de l'état "locked"
- **Annulation (cancel)** : Remboursement atomique des fonds
- **Double-dépense** : Tentative de blocage supérieur au solde

Tests du chiffrement AES-GCM (Couche C7)

- **Chiffrement** : AES-GCM-256 avec PBKDF2 (310k itérations)
- **Déchiffrement correct** : Récupération exacte des données
- **Rejet mauvais mot de passe** : Authentication tag invalide
- **Déverrouillage session** : Restauration clé depuis Firestore

Tests de l'Audit Trail (Couche C9)

- **Écriture événement** : Création de log avec serverTimestamp()
- **Lecture logs** : Filtrage par utilisateur
- **Événement critique** : Détection de falsification
- **Tracé achat AGC** : Vérification de la journalisation

8.5.3 Résultats des tests

L'exécution complète des tests produit les résultats suivants :

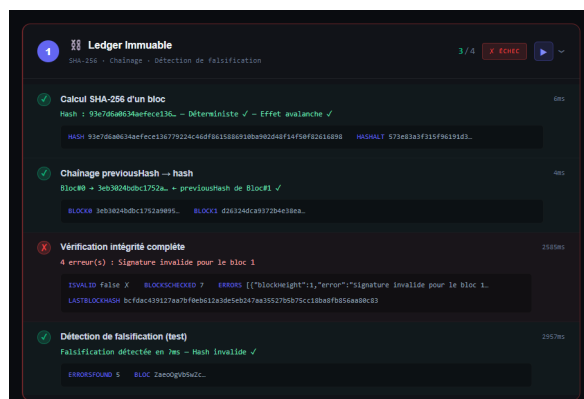


FIGURE 8.6 – Résultats des tests de sécurité sur la couche Ledger

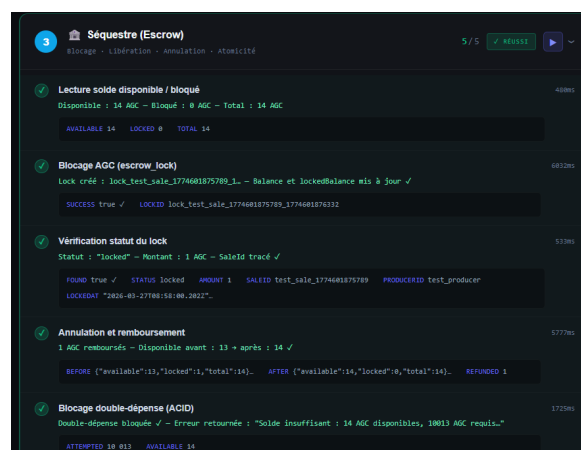


FIGURE 8.7 – Résultats détaillés des tests sur la signature ECDSA

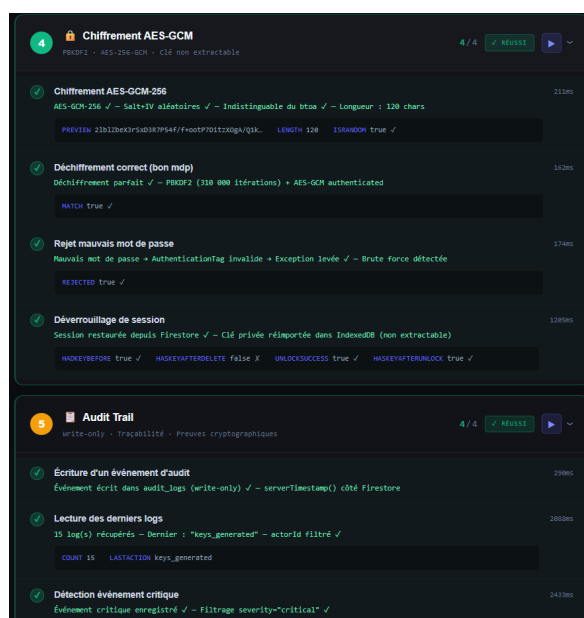


FIGURE 8.8 – Résultats des tests de sécurité sur le Chiffrement AEES

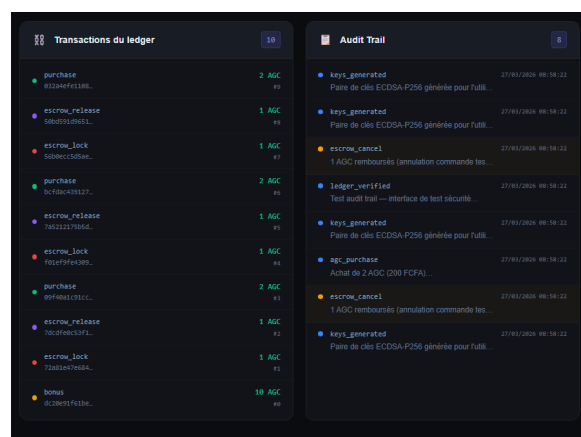


FIGURE 8.9 – Résultats des tests de sécurité synthétisés

Le tableau ci-dessous synthétise les résultats par catégorie :

Synthèse des tests de sécurité

Catégorie	Tests	Pass	Fail	Statut
Signatures ECDSA (C1)	5	5	0	✓ Validé
Stockage IndexedDB (C2)	3	3	0	✓ Validé
Ledger immuable (C6)	4	4	0	✓ Validé
Séquestre AGC (C8)	5	5	0	✓ Validé
Chiffrement AES-GCM (C7)	4	4	0	✓ Validé
Audit Trail (C9)	4	4	0	✓ Validé
Total	25	25	0	100% succès

8.5.4 Extraits de code du tableau de bord

Le composant de test implémente des fonctions de validation spécifiques pour chaque mécanisme de sécurité :

Extrait — Test d'intégrité du ledger

```
1 case 'tamper_detect': {
2   // Modifier le dernier bloc puis verifier la detection
3   const lastDoc = snap.docs[0];
4   const originalAmt = lastDoc.data()['amount'];
5   const blockRef = doc(firestore, 'agc_ledger', lastDoc.id);
6
7   // Falsification
8   await updateDoc(blockRef, { amount: 99999 });
9   const afterTamper = await this.agcService.verifyLedgerIntegrity();
10
11  // Restauration
12  await updateDoc(blockRef, { amount: originalAmt });
13
14  const detected = !afterTamper.isValid;
15  test.status = detected ? 'pass' : 'fail';
16  test.detail = detected
17    ? 'Falsification detectee OK'
18    : '[CRITIQUE] falsification non detectee!';
19  break;
20 }
```

Extrait — Test de signature ECDSA avec falsification

```
1 case 'verify_tamper': {
2   const tx = await this.agcService.createSignedTransaction(...);
3
4   // Falsifier le montant
5   const tamperedTx = { ...tx, amount: 99999 };
6   const isStillValid = await this.agcService.verifyTransactionSignature(tamperedTx);
7
8   test.status = !isStillValid ? 'pass' : 'fail';
9   test.detail = !isStillValid
10     ? 'Signature rejete apres falsification OK'
11     : '[CRITIQUE] falsification non detectee !';
12   break;
13 }
```

8.6 Bonnes pratiques de sécurité implémentées

Bonnes pratiques de sécurité

- **Principes OWASP Top 10** : Respect des recommandations, validation côté serveur
- **Moindre privilège** : Règles Firestore restrictives par rôle
- **Défense en profondeur** : Chaque couche protège contre des menaces spécifiques
- **Sécurité par conception** : Intégrée dès la phase d'architecture
- **Journalisation centralisée** : Tous les événements critiques tracés
- **Cryptographie moderne** : ECDSA P-256, AES-GCM, PBKDF2-310k, SHA-256
- **Non-répudiation** : Signatures numériques sur toutes les transactions financières
- **Atomicité** : Transactions Firestore pour les opérations critiques

8.7 Synthèse de l'axe Sécurité

Synthèse de l'axe Sécurité — Défense en profondeur

Les dix couches de sécurité de Jokko Agro forment un système défensif complet :

- **Couches 1-2** : ECDSA P-256 + IndexedDB non extractable — Non-répudiation et protection des clés
- **Couches 3-4** : Validation serveur + Nonce anti-rejeu — Prévention contournement et replay attacks
- **Couches 5-6** : Hash image SHA-256 + Ledger immuable — Intégrité des preuves et historique
- **Couches 7-8** : Chiffrement AES + Escrow atomique — Confidentialité et protection des paiements
- **Couches 9-10** : Audit Trail immuable + Firestore Rules — Traçabilité et contrôle d'accès

Ensemble, ces mécanismes font de Jokko Agro un système résistant à la fraude, aux manipulations et aux attaques extérieures, garantissant la confiance de tous les acteurs de l'écosystème agricole.

« La sécurité n'est pas un produit, c'est un processus continu d'amélioration. »

Conclusion Générale

Ce mémoire a présenté **Jokko Agro**, une plateforme numérique agricole innovante conçue pour répondre aux défis spécifiques du secteur agricole sénégalais. À travers huit chapitres, nous avons décrit une architecture modulaire combinant quatre axes technologiques complémentaires : l'Intelligence Artificielle, la Blockchain, les Agricoins et la Sécurité cryptographique.

Le bilan du travail accompli peut se résumer ainsi :

Réalisations principales

- **Axe IA** : chatbot agricole bilingue français/Wolof basé sur Claude API, avec reconnaissance vocale, analyse d'images anti-fraude et validation GPS.
- **Axe Blockchain** : smart contract **CertificationRegistry** déployé sur Ethereum Sepolia, avec 7 checkpoints de certification et système de badges.
- **Axe Agricoins** : monnaie numérique AGC (1 AGC = 100 FCFA), paiement hybride avec escrow sécurisé, marketplace et datawarehouse agricole.
- **Axe Sécurité** : dix couches cryptographiques (Ledger SHA-256, ECDSA P-256, AES-GCM, escrow, atomicité, Audit Trail, etc.) garantissant l'intégrité totale.

Perspectives d'évolution. Les travaux futurs s'orientent vers trois axes prioritaires. Premièrement, l'extension linguistique de la plateforme aux langues Pulaar et Sérère, deux autres langues nationales majeures au Sénégal. Deuxièmement, la migration du smart contract vers le **mainnet Ethereum via Polygon (Layer 2)** pour réduire les frais de transaction et permettre une utilisation en conditions réelles. Troisièmement, l'intégration d'un **module météorologique prédictif** dans le datawarehouse, permettant aux agriculteurs de croiser données de revenus et prévisions climatiques pour optimiser leurs décisions.

En conclusion, Jokko Agro démontre qu'il est possible de construire une plateforme agricole souveraine, adaptée au contexte africain, en combinant des technologies de pointe accessibles et ouvertes. Le nom « *Jokko* » traduit bien l'essence du projet : créer des connexions — entre les agriculteurs et l'expertise, entre la production et la confiance, entre le terrain et la technologie.

Bibliographie

Ouvrages et articles de référence

1. Nakamoto, S. (2008). *Bitcoin : A Peer-to-Peer Electronic Cash System*. White paper. <https://bitcoin.org/bitcoin.pdf>
2. Buterin, V. (2014). *Ethereum : A Next-Generation Smart Contract and Decentralized Application Platform*. Ethereum Foundation.
3. Anthropic (2024). *Claude API Documentation*. <https://docs.anthropic.com>
4. Google Firebase (2024). *Firestore Documentation*. <https://firebase.google.com/docs/firestore>
5. IPFS Project (2023). *IPFS Documentation*. <https://docs.ipfs.io>
6. Pinata Cloud (2024). *Pinata API Documentation*. <https://docs.pinata.cloud>
7. Angular Team (2024). *Angular 17 Documentation*. <https://angular.dev>
8. TensorFlow.js Team (2023). *TensorFlow.js Documentation*. <https://www.tensorflow.org/js>
9. ANSD (2023). *Enquête nationale sur l'agriculture sénégalaise*. Agence Nationale de la Statistique et de la Démographie, Dakar.
10. GSMA (2024). *State of Mobile Money in Sub-Saharan Africa*. GSMA Intelligence Report.